



6장. 자원 축소

4팀 – 최혜윤, 김태영, 정윤지

목차

#01 6.1 차원 축소 개요 ~ 6.3 LDA

#02 6.4 SVD ~ 6.5 NMF

#03 다른 차원축소 기법들
& Kaggle(Dimensionality Reduction for Beginners)



6.1 Dimension Reduction 개요



6.1 Dimension Reduction 개요

Dimension Reduction

: 많은 피처로 구성된 다차원 데이터 세트의 차원 축소 → 새로운 차원의 데이터 세트 생성

If 차원 증가 → 데이터 point 간 거리 기하급수적으로 증가!

⇒ sparse(희소한) 구조

⇒ 차원의 저주!

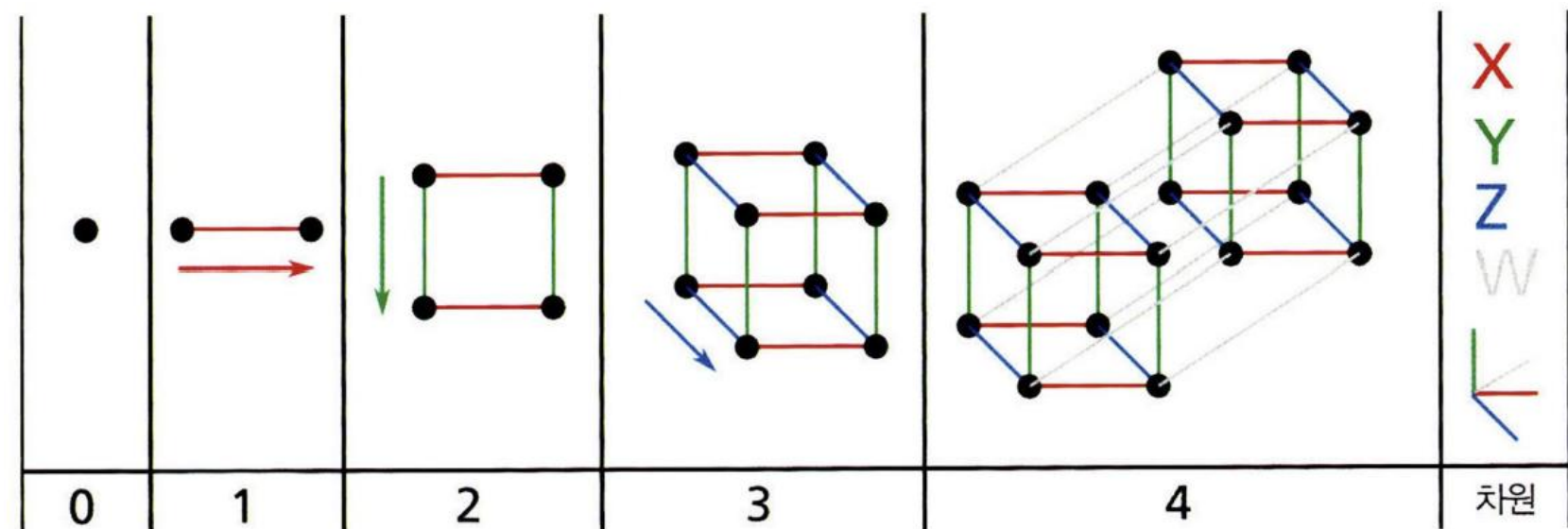
6.1 Dimension Reduction 개요

차원의 저주

: 데이터 학습을 위해 차원이 증가 → 학습데이터 수 < 차원의 수 → 성능 저하!

(sparse(희소) 현상 : 데이터 point 간의 거리가 기하급수적으로 늘어남)

⇒ 정보의 수가 적음 → 성능 저하 (과대적합 위험 증가)



6.1 Dimension Reduction 개요

차원의 저주

: 데이터 학습을 위해 차원이 증가 → 학습데이터 수 < 차원의 수 → 성능 저하!

(sparse(희소) 현상 : 데이터 point 간의 거리가 기하급수적으로 늘어남)

[단점]

1. 수백 개 이상 피처로 구성된 데이터 세트 → 예측 신뢰도 저하
2. 피처 수 증가 → 입력 변수 간 상관관계 높음
⇒ 다중 공선성 문제 → 예측 성능 저하!

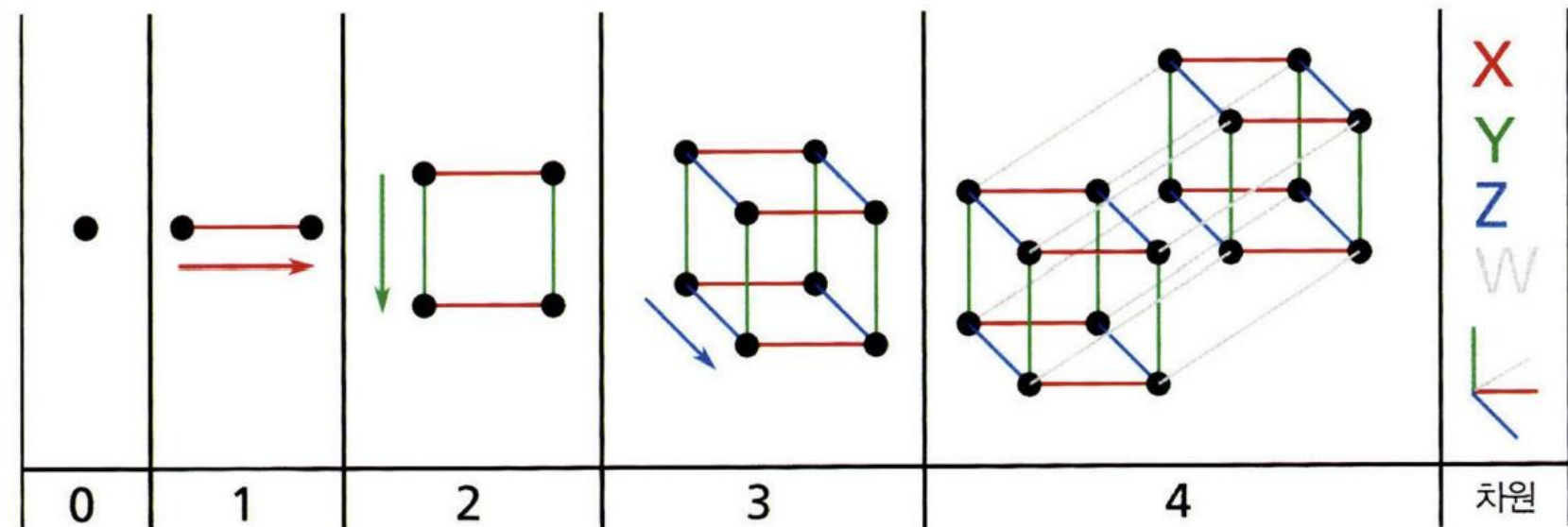
6.1 Dimension Reduction 개요

차원의 저주

[해결 방법]

훈련 세트 크기 키우기 → 훈련 샘플 밀도 증가

But, 필요한 훈련 샘플 수 : 차원 수 증가함에 따라 기하급수적으로 증가!

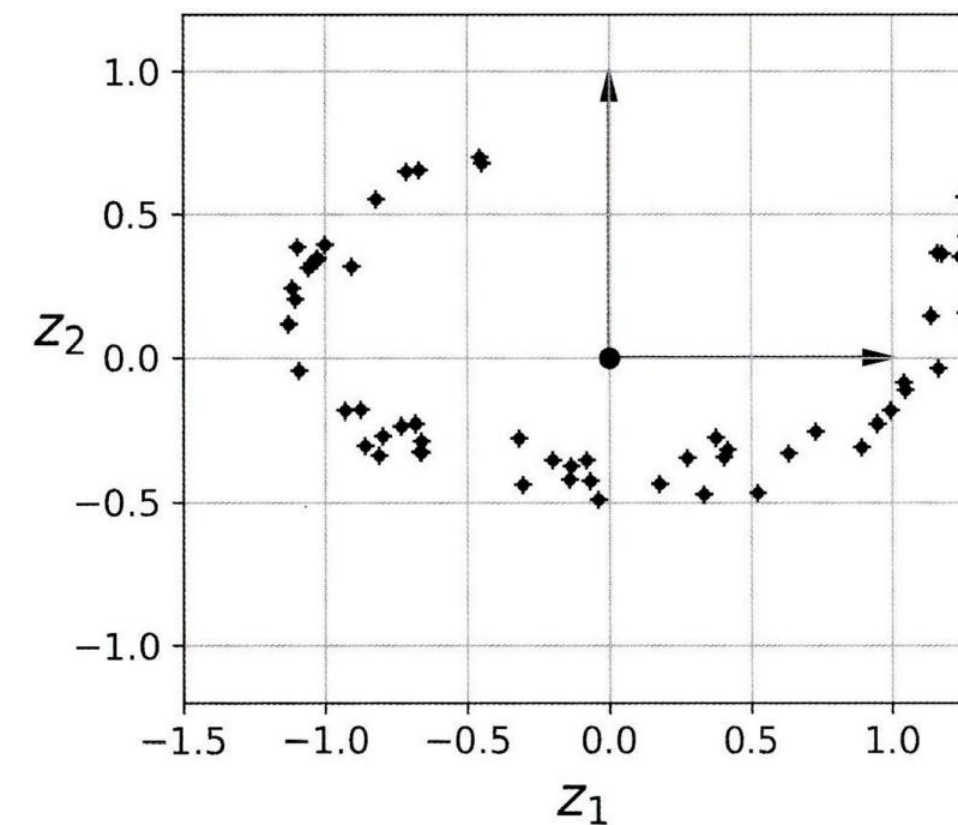
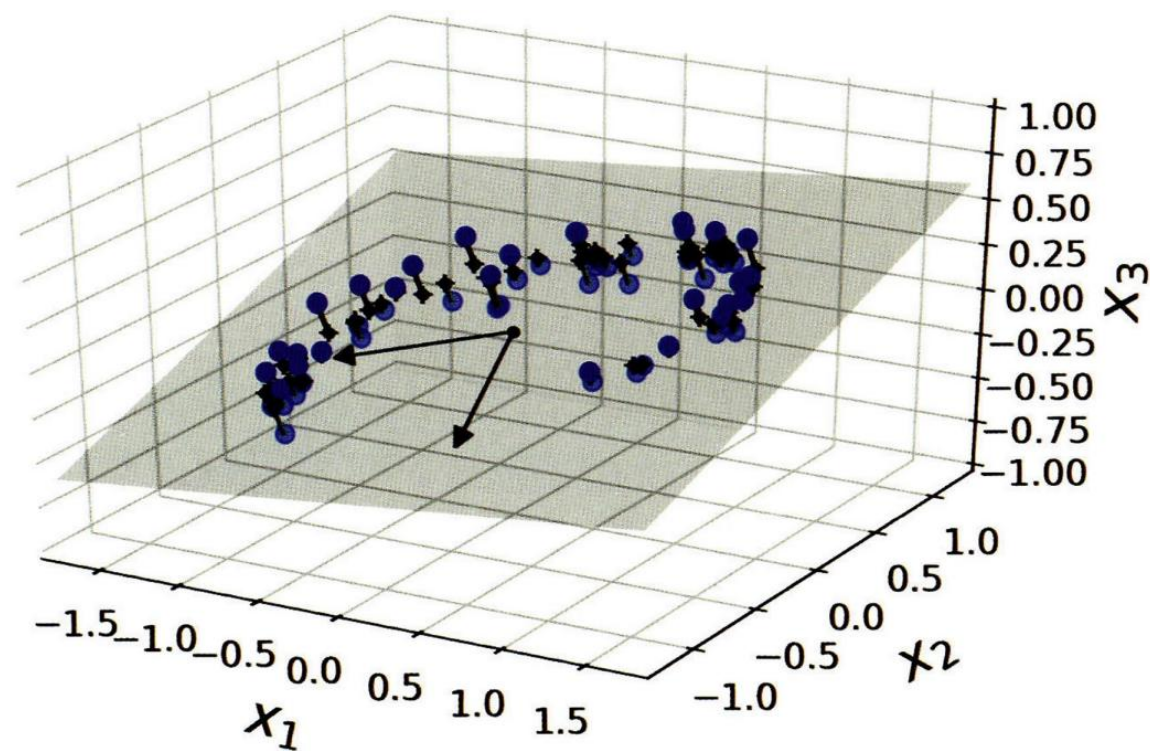


6.1 Dimension Reduction 개요

차원 축소를 위한 접근 방법

① 투영

“모든 훈련 샘플이 고차원 안의 저차원 부분 공간에 놓여 있다”



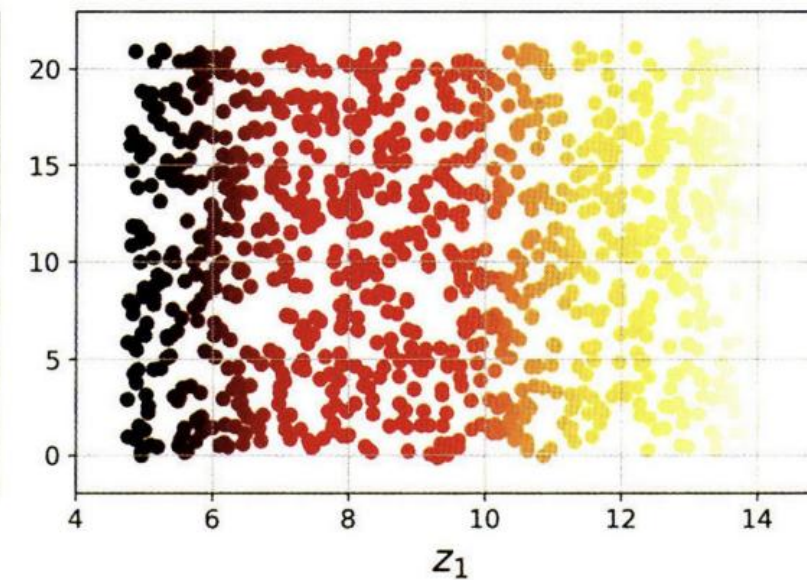
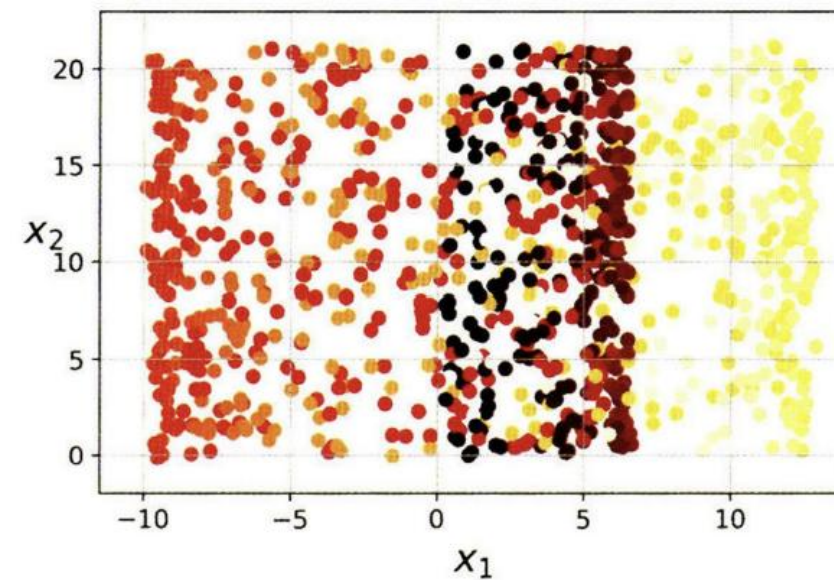
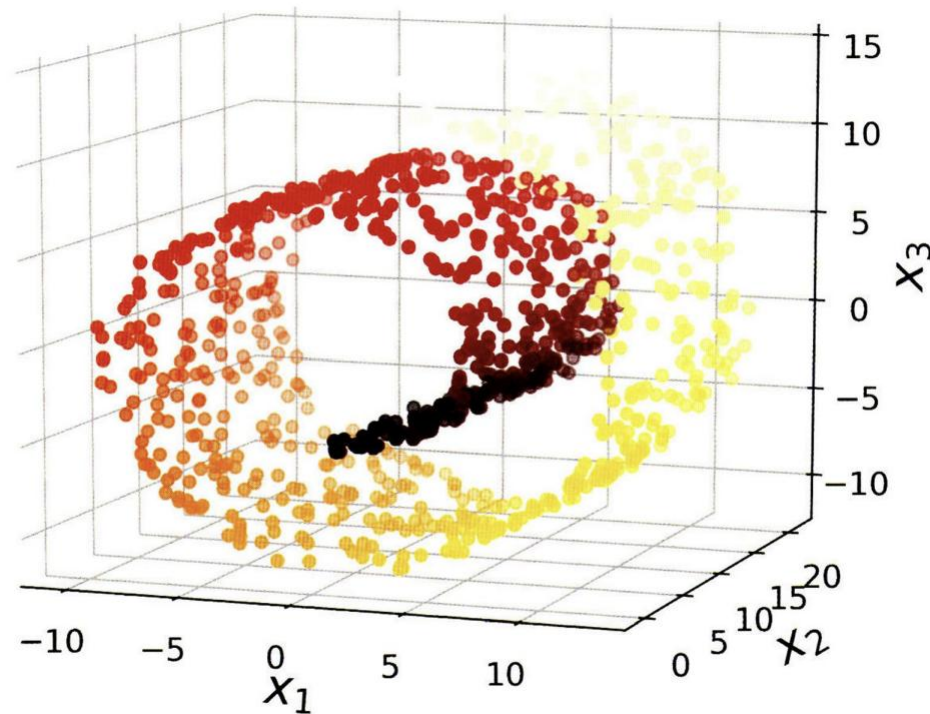
6.1 Dimension Reduction 개요

차원 축소를 위한 접근 방법

② 매니폴드 학습

스위스 롤 (2D 매니폴드)

: 고차원 공간에서 휘어지거나 뒤틀린 2D 모양

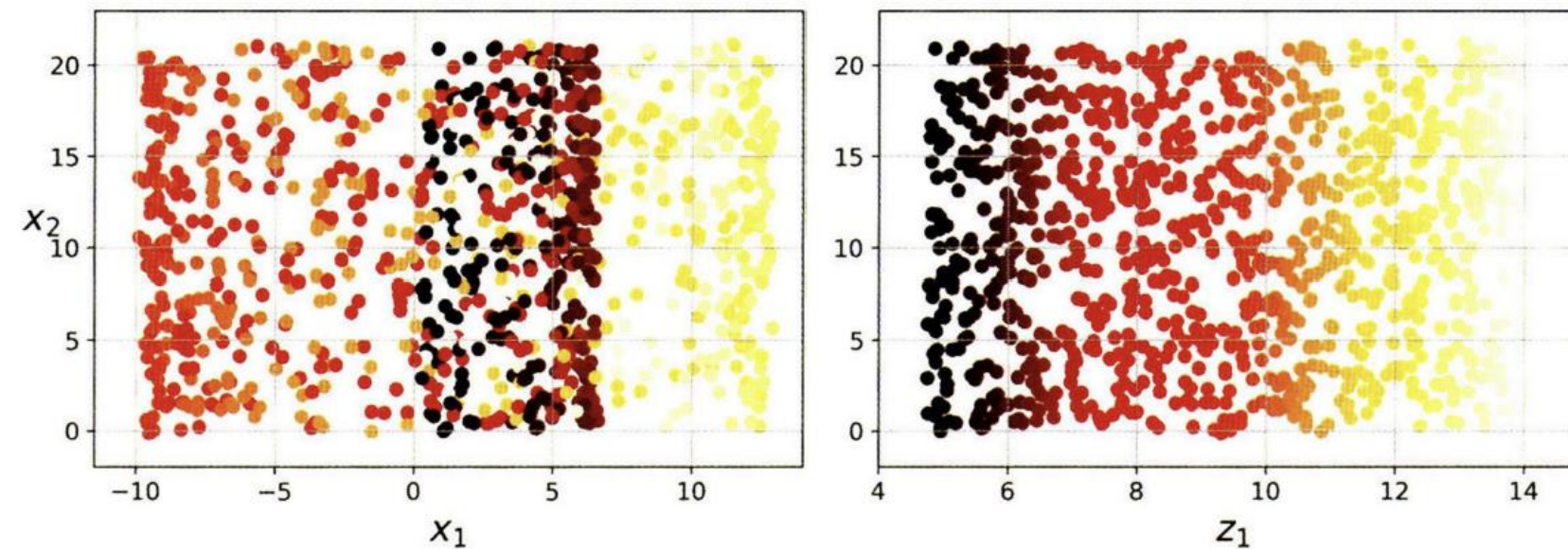
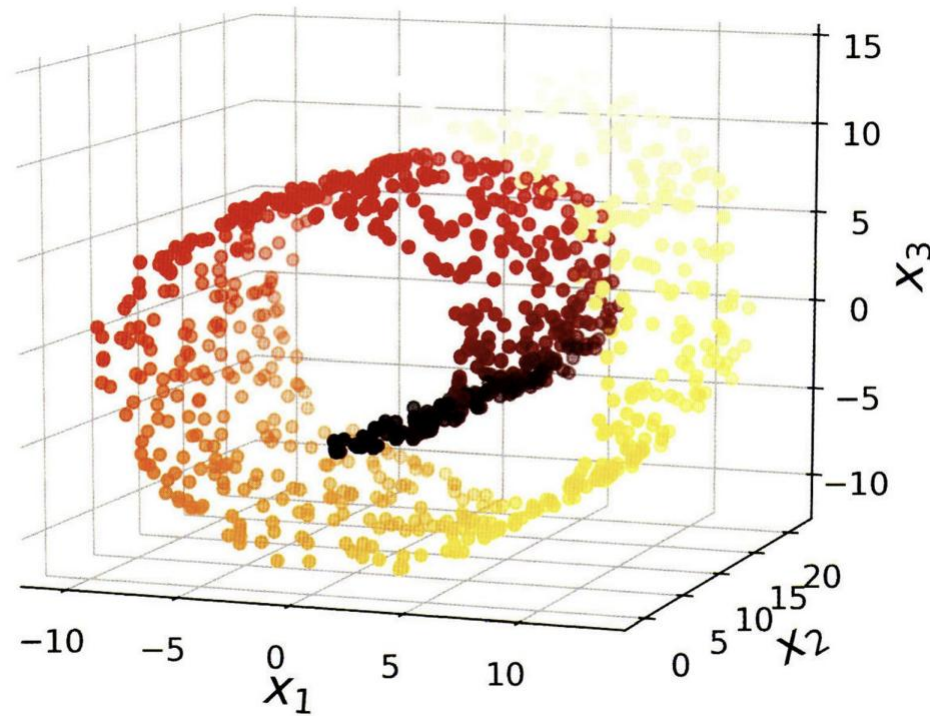


6.1 Dimension Reduction 개요

차원 축소를 위한 접근 방법

② 매니폴드 학습

d차원 매니폴드 : d차원 초평면으로 보일 수 있는 n차원 공간의 일부 ($d < n$)



6.1 Dimension Reduction 개요

차원 축소를 위한 접근 방법

② 매니폴드 학습

매니폴드 학습 : 많은 축소 알고리즘이 훈련 샘플이 놓여 있는 매니폴드를 모델링

매니폴드 가정(가설) : 실제 고차원 데이터셋이 더 낮은 저차원 매니폴드에 가깝게 놓여있음

=> 바로 처리해야 할 작업이 저차원 매니폴드 공간에 표현되면 더 간단해질 것 !

6.1 Dimension Reduction 개요

차원 축소를 위한 접근 방법

② 매니폴드 학습

매니폴드 학습 : 많은 축소 알고리즘이 훈련 샘플이 놓여 있는 매니폴드를 모델링

매니폴드 가정(가설) : 실제 고차원 데이터셋이 더 낮은 저차원 매니폴드에 가깝게 놓여있음

But, 이런 가정이 항상 유효하지 X !

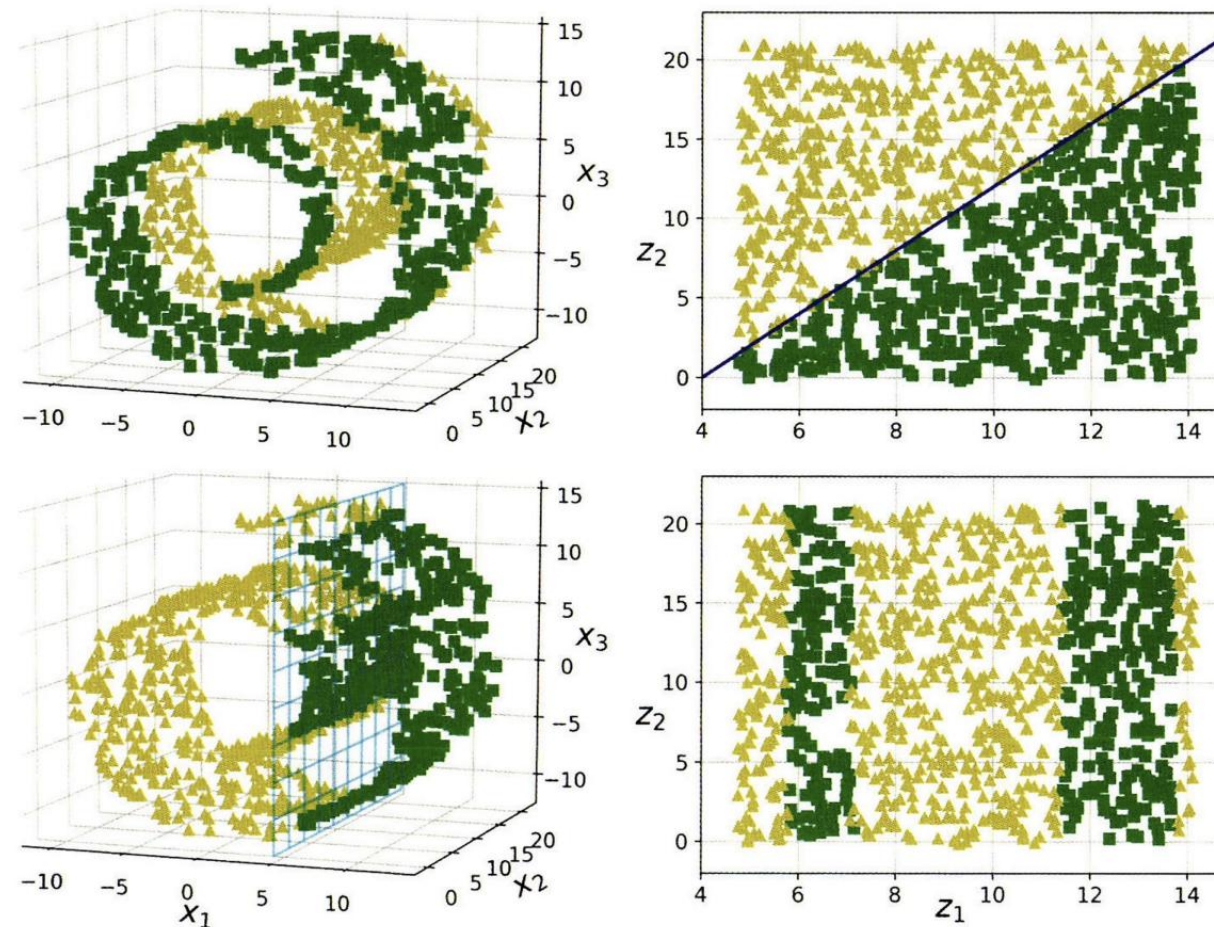
6.1 Dimension Reduction 개요

차원 축소를 위한 접근 방법

② 매니폴드 학습

모델을 훈련시키기 전에 훈련 세트의 차원을 감소시키면 훈련 속도는 빨라지지만, 항상 더 낮거나 간단한 솔루션이 되는 것은 X!

→ 데이터셋에 따라 달라짐!



6.1 Dimension Reduction 개요

Dimension Reduction 장점

1. 차원 축소를 통해 피쳐 줄이기 → 직관적으로 데이터 해석 가능
2. 학습 데이터 크기 축소 → 학습에 필요한 처리 능력 감소

6.1 Dimension Reduction 개요

차원 축소

① 피처 선택 (feature selection)

: 종속성이 강한 불필요한 피처는 제거, 데이터 특징 잘 나타내는 주요 피처만 선택

② 피처 추출 (feature extraction)

: 기존 피처를 저차원의 중요 피처로 압축해서 추출하는 것

→ 기존의 피처와 완전히 다른 값

→ 단순 압축 X, 피처를 함축적으로 잘 설명할 수 있는 다른 공간으로 매핑해 추출

→ 기존 피처가 인지 불가능했던 잠재적인 요소(Latent Factor) 추출

6.1 Dimension Reduction 개요

차원 축소 기능

① 잠재된 특성을 피처로 도출해 함축적 형태의 이미지 변환과 압축

→ 작은 차원 → 분류 수행 시 과적합(overfitting) 영향 작음 → 예측 성능 증가

Ex) PCA

② 텍스트의 숨겨진 의미 추출

→ 문서 내 숨겨져 있는 시맨틱(Semantic) 의미 or 토픽(Topic)을 잠재 요소로 간주 → 추출

Ex) SVD, NMF

6.2 PCA(Principal Component Analysis)

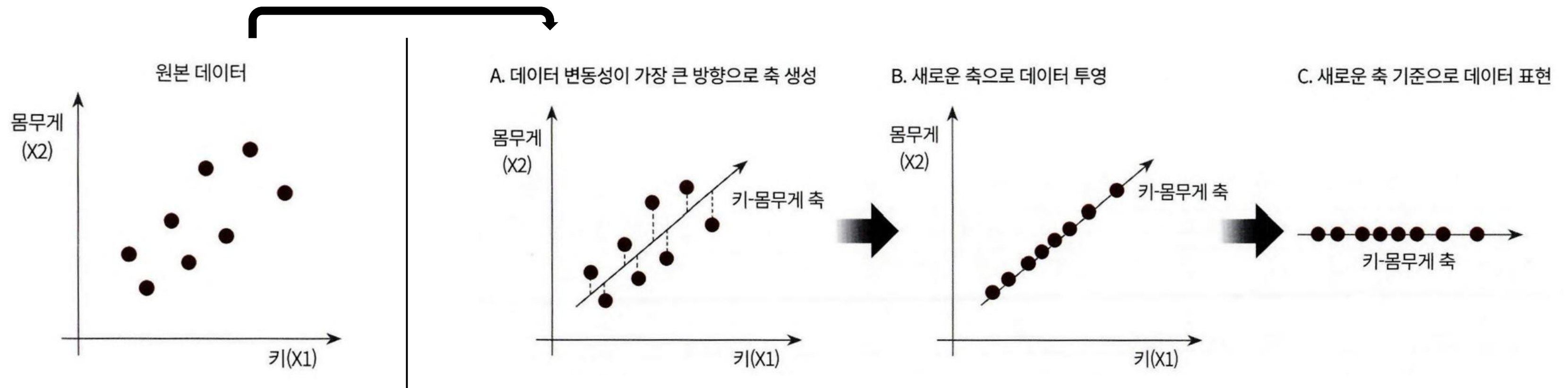


6.2 PCA(Principal Component Analysis)

PCA 개요

여러 변수 간의 상관관계 이용 → 주성분 (Principal Component) 추출 → 차원 축소

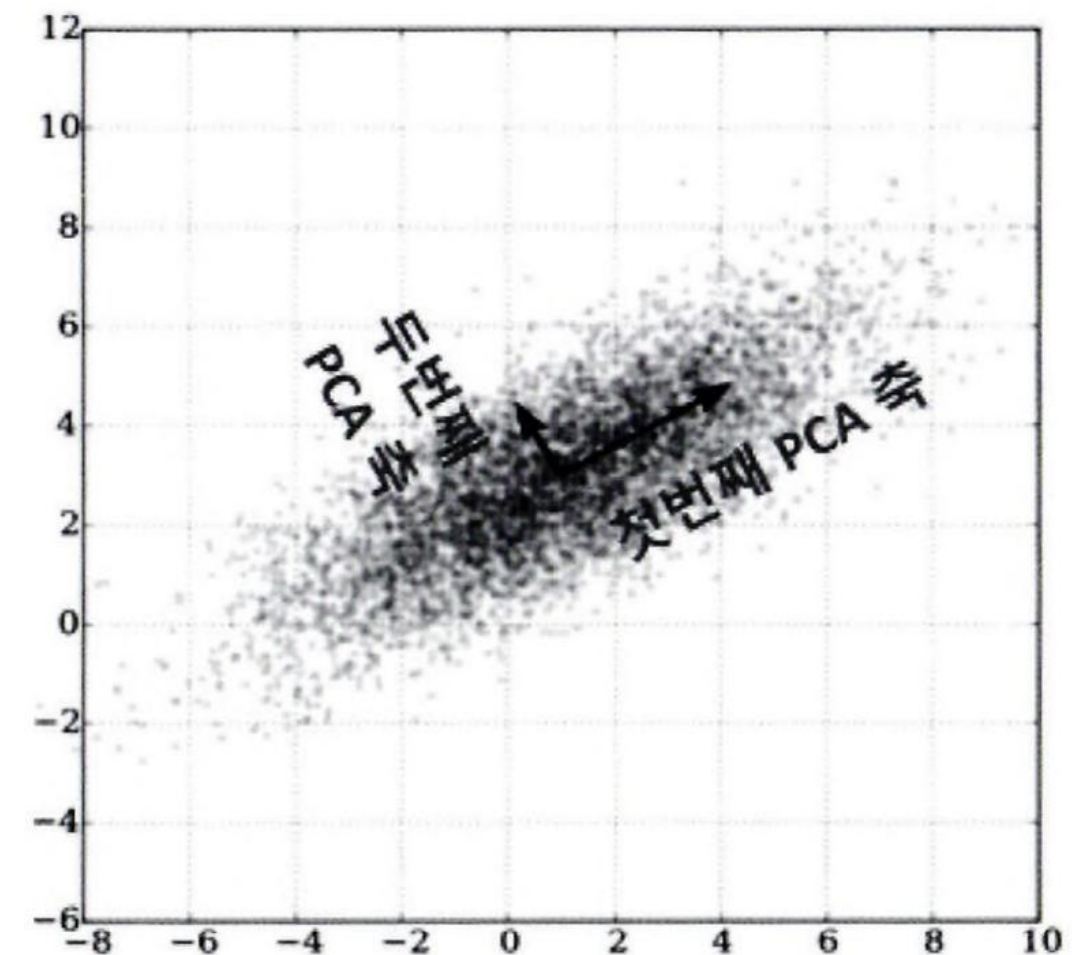
PCA 주성분 : 가장 높은 분산을 가지는 데이터
(“**분산**”이 데이터 특성을 가장 잘 나타냄)



6.2 PCA(Principal Component Analysis)

PCA 과정

- ① 첫 번째 벡터 축 생성 (가장 큰 데이터 변동성(Variance) 기반)
- ② 두 번째 벡터 축 생성 (첫 번째 축에 대한 직교 벡터 기반)
- ③ 세 번째 벡터 축 생성 (두 번째 축에 대한 직교 벡터 기반)
- ④ 생성된 벡터 축에 원본 데이터 투영
→ 벡터 축 개수만큼의 차원으로 축소



6.2 PCA(Principal Component Analysis)

PCA 개념

: 원본 데이터의 피처 개수에 비해 매우 작은 주성분으로
원본 데이터의 총 변동성을 설명할 수 있는 분석법

[선형대수 관점 해석]

- 입력 데이터의 공분산 행렬(Covariance Matrix)를 고윳값 분해
- 구한 고윳값 벡터에 입력 데이터를 선형 변환

** 고유벡터, 고윳값, 선형 변환, 공분산 행렬...

6.2 PCA(Principal Component Analysis)

PCA 개념

: 원본 데이터의 피처 개수에 비해 매우 작은 주성분으로
원본 데이터의 총 변동성을 설명할 수 있는 분석법

[선형대수 관점 해석]

- 입력 데이터의 공분산 행렬(Covariance Matrix)를 고윳값 분해
- 구한 고윳값 벡터에 입력 데이터를 선형 변환

=> PCA

6.2 PCA(Principal Component Analysis)

PCA 과정

1. 입력 데이터 세트의 공분산 행렬 생성
2. 공분산 행렬의 고유벡터 & 고유값 계산
3. 고유값이 가장 큰 순으로 K개의 고유벡터 추출
4. 추출된 고유벡터 이용해 입력 데이터를 변환

6.2 PCA(Principal Component Analysis)

+ 적절한 차원의 수 선택하기

충분한 분산(ex. 95%)이 될 때 까지 더해야 할 차원 수 선택하기

1. 차원 축소 X, PCA 계산 후 훈련 세트 분산 95%로 유지하는 데 필요한 최소한의 차원 수 계산

```
pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
```

n_components=d로 설정 후, PCA 다시 실행
(분산의 비율을 0.0 ~ 1.0으로 설정)

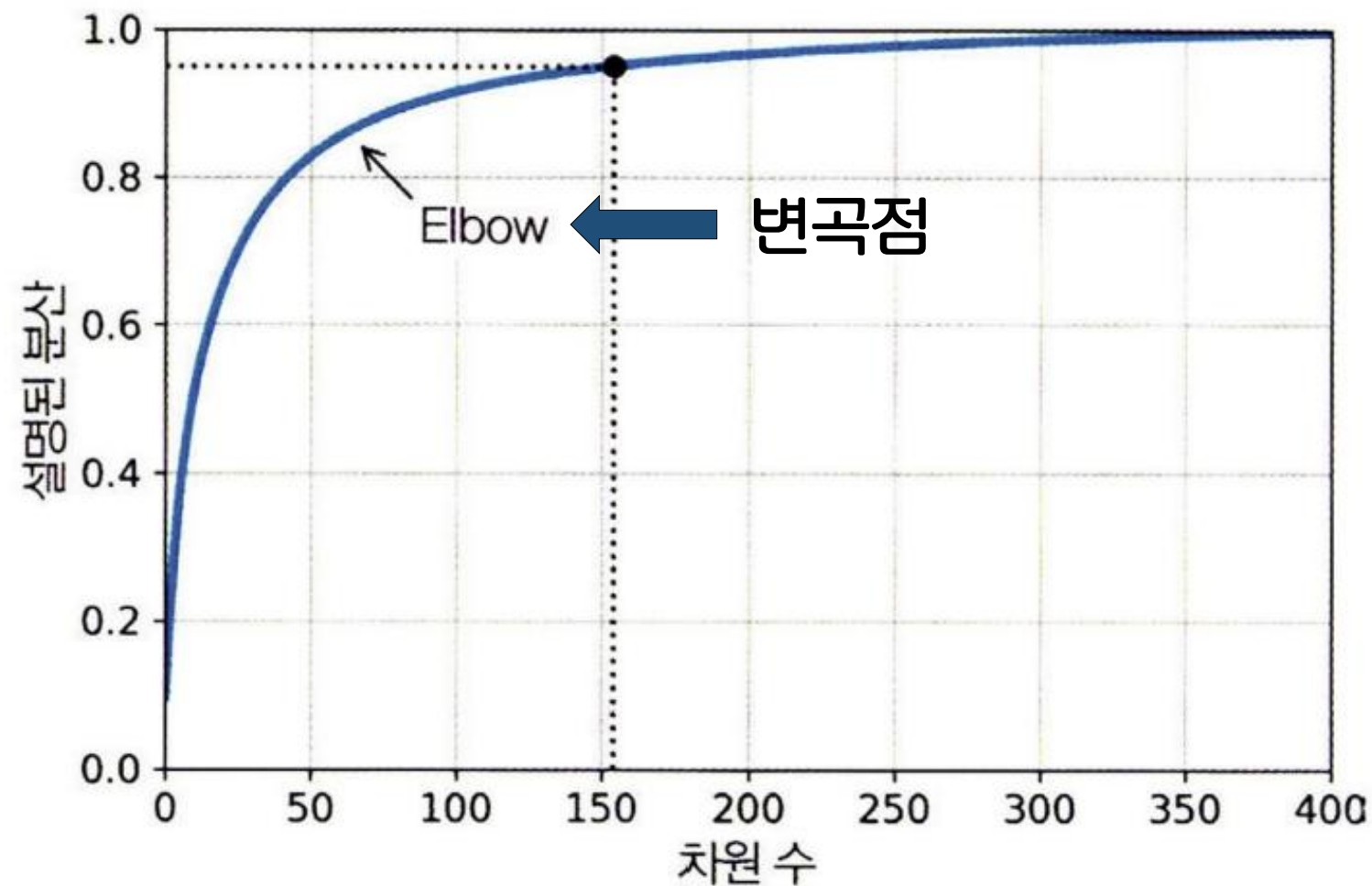
```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

6.2 PCA(Principal Component Analysis)

+ 적절한 차원의 수 선택하기

충분한 분산(ex. 95%)이 될 때 까지 더해야 할 차원 수 선택하기

2. 분산을 차원 수에 대한 함수로 그리는 것.



6.2 PCA(Principal Component Analysis)

[PCA 실습 - 붓꽃(Iris) 데이터 세트]

사이킷런 붓꽃 데이터 로딩 -> DataFrame으로 변환

```
[1] from sklearn.datasets import load_iris
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

iris = load_iris()
# 넘파이 데이터 세트를 판다스 DataFrame으로 변환
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
irisDF = pd.DataFrame(iris.data, columns=columns)
irisDF['target'] = iris.target
irisDF.head(3)
```



	sepal_length	sepal_width	petal_length	petal_width	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0



6.2 PCA(Principal Component Analysis)

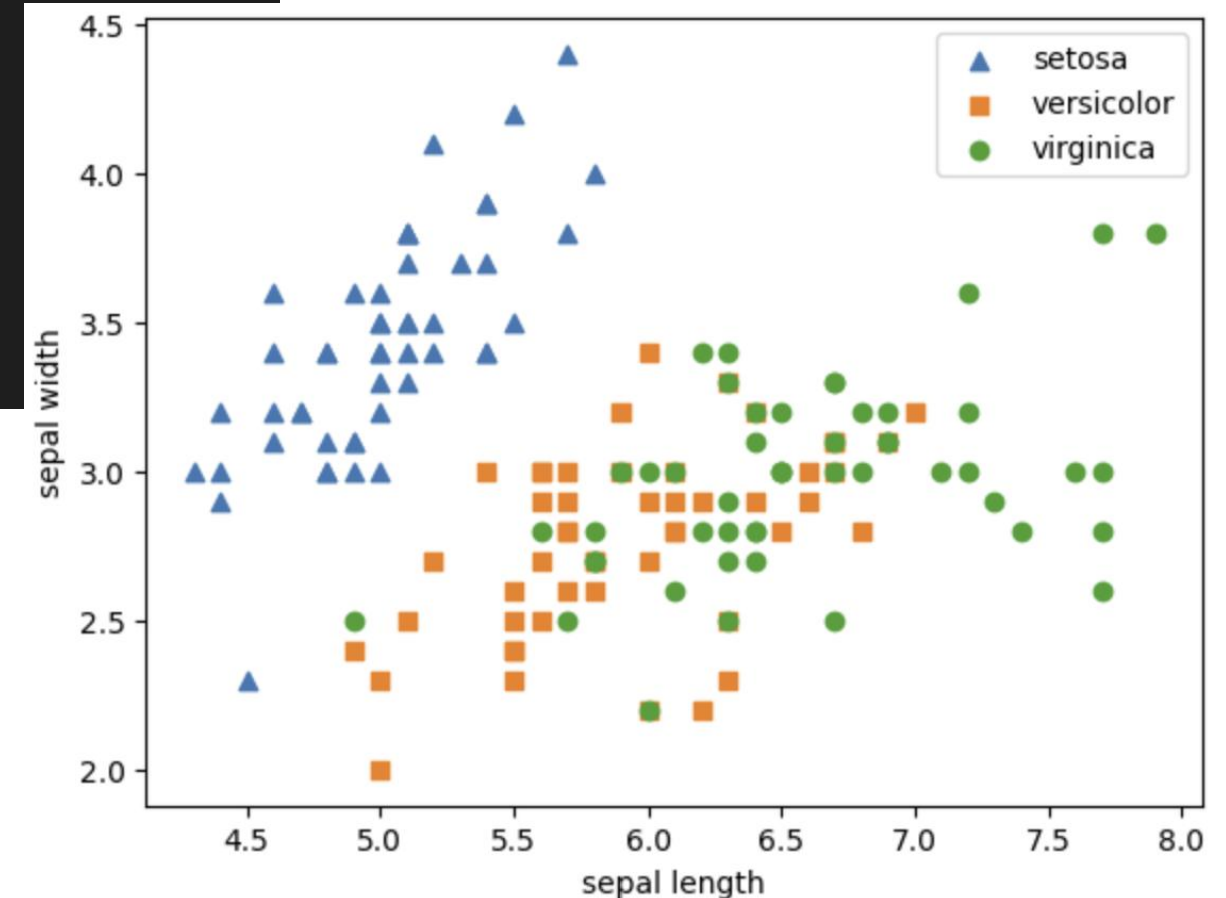
[PCA 실습 - 붓꽃(Iris) 데이터 세트]

붓꽃 데이터 세트 분포 2차원 시각화

```
[3] #setosa는 세모, versicolor는 네모, virginica는 동그라미로 표현
    markers=['^', 's', 'o']

    #setosa의 target 값은 0, versicolor는 1, virginica는 2. 각 target별로 다른 모양으로 산점도로 표시
    for i, marker in enumerate(markers):
        x_axis_data = irisDF[irisDF['target']==i] ['sepal_length']
        y_axis_data = irisDF[irisDF['target']==i] ['sepal_width']
        plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

    plt.legend()
    plt.xlabel('sepal length')
    plt.ylabel('sepal width')
    plt.show()
```



6.2 PCA(Principal Component Analysis)

[PCA 실습 - 붓꽃(Iris) 데이터 세트]

PCA 적용 전, 개별 속성 스케일링

```
[4] from sklearn.preprocessing import StandardScaler

# Target 값을 제외한 모든 속성 값을 StandardScaler를 이용해 표준 정규 분포를 가지는 값들로 변환
iris_scaled = StandardScaler().fit_transform(irisDF.iloc[:, :-1])
```

PCA 적용 -> 4차원 붓꽃 데이터를 2차원 PCA 데이터로 변환

```
[5] from sklearn.decomposition import PCA

pca = PCA(n_components=2)

# fit()과 transform()을 호출해 PCA 변환 데이터 반환
pca.fit(iris_scaled)
iris_pca = pca.transform(iris_scaled)
print(iris_pca.shape)
```

➡ (150, 2)

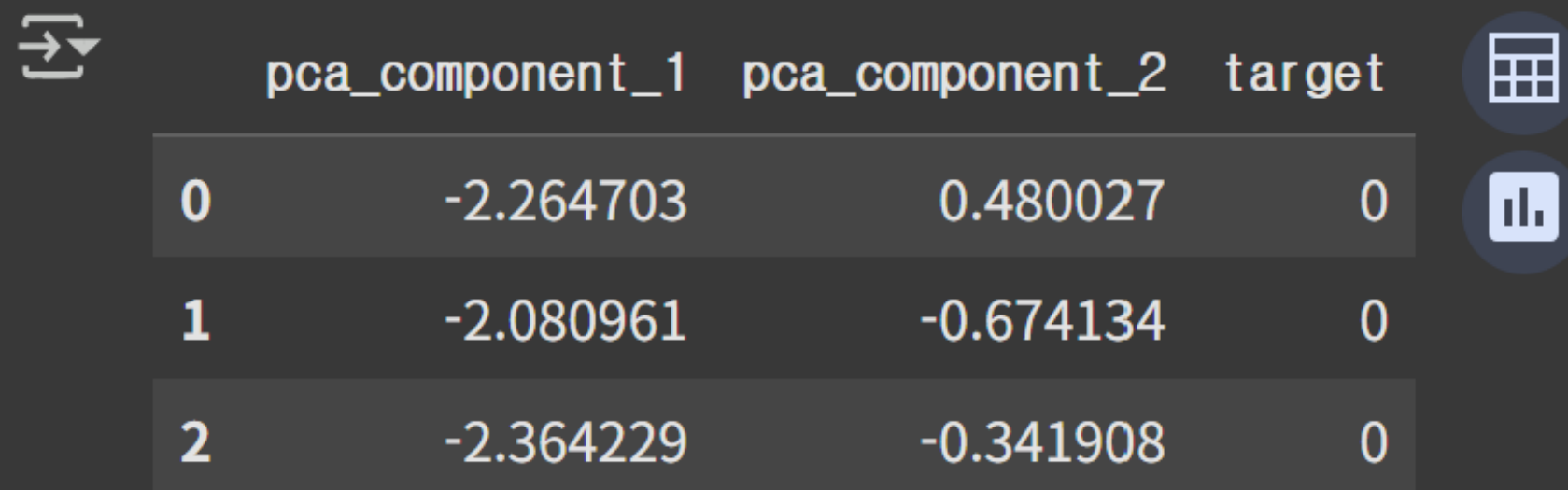
6.2 PCA(Principal Component Analysis)

[PCA 실습 - 붓꽃(Iris) 데이터 세트]

변환된 PCA 데이터 세트를 DataFrame으로 변환 -> 데이터값 확인

```
[6] # PCA 변환된 데이터의 컬럼명을 각각 pca_component_1, pca_component_2로 명명
```

```
pca_columns=['pca_component_1','pca_component_2']  
irisDF_pca = pd.DataFrame(iris_pca, columns=pca_columns)  
irisDF_pca['target']=iris.target  
irisDF_pca.head(3)
```



	pca_component_1	pca_component_2	target
0	-2.264703	0.480027	0
1	-2.080961	-0.674134	0
2	-2.364229	-0.341908	0

6.2 PCA(Principal Component Analysis)

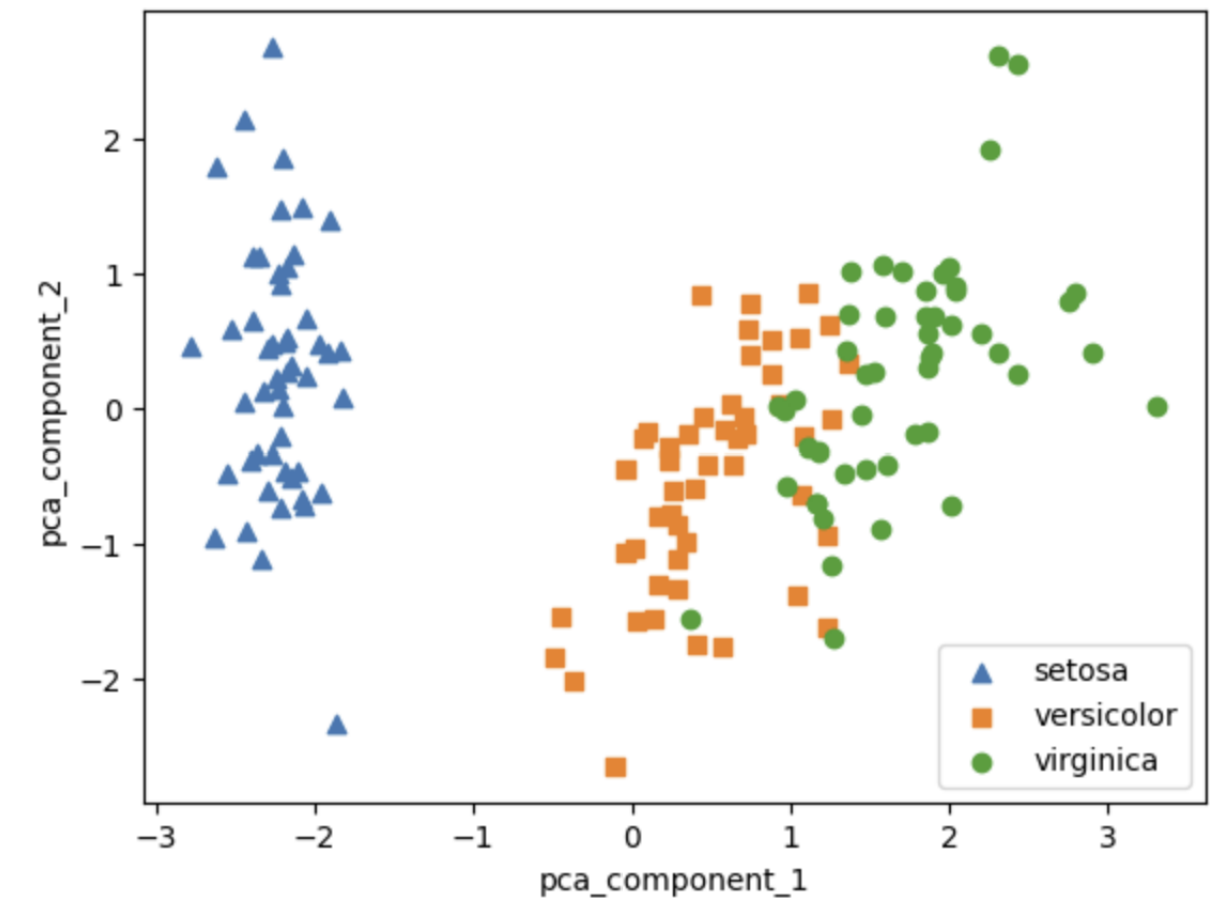
[PCA 실습 - 붓꽃(Iris) 데이터 세트]

PCA 변환된 데이터 세트 2차원 시각화

```
[7] # setosa를 세모, versicolor를 네모, virginica를 동그라미로 표시
     markers=['^', 's', 'o']

     # pca_component_1 을 x축, pc_component_2를 y축으로 scatter plot 수행
     for i, marker in enumerate(markers):
         x_axis_data = irisDF_pca[irisDF_pca['target']==i]['pca_component_1']
         y_axis_data = irisDF_pca[irisDF_pca['target']==i]['pca_component_2']
         plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

     plt.legend()
     plt.xlabel('pca_component_1')
     plt.ylabel('pca_component_2')
     plt.show()
```



PCA Component별 원본 데이터 변동성 반영률 확인

```
print(pca.explained_variance_ratio_)
```

```
[0.72962445 0.22850762]
```

6.2 PCA(Principal Component Analysis)

[PCA 실습 - 붓꽃(Iris) 데이터 세트]

{원본 붓꽃 데이터 세트 & PCA로 변환된 데이터 세트에 각각 분류 적용}

1. 원본 붓꽃 데이터에 Random Forest 적용

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

rcf = RandomForestClassifier(random_state=156)
scores = cross_val_score(rcf, iris.data, iris.target, scoring='accuracy', cv=3)
print('원본 데이터 교차 검증 개별 정확도:', scores)
print('원본 데이터 평균 정확도:', np.mean(scores))
```

원본 데이터 교차 검증 개별 정확도: [0.98 0.94 0.96]
원본 데이터 평균 정확도: 0.96

2 . PCA로 변환된 데이터 세트에 Random Forest 적용

```
[11] pca_X = irisDF_pca[['pca_component_1', 'pca_component_2']]
scores_pca = cross_val_score(rcf, pca_X, iris.target, scoring='accuracy', cv=3)
print('PCA 변환 데이터 교차 검증 개별 정확도:', scores_pca)
print('PCA 변환 데이터 평균 정확도:', np.mean(scores_pca))
```

PCA 변환 데이터 교차 검증 개별 정확도: [0.88 0.88 0.88]
PCA 변환 데이터 평균 정확도: 0.88

6.2 PCA(Principal Component Analysis)

[PCA 실습 - 붓꽃(Iris) 데이터 세트]

{원본 붓꽃 데이터 세트 & PCA로 변환된 데이터 세트에 각각 분류 적용}

1. 원본 붓꽃 데이터에 Random Forest 적용

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

rfr = RandomForestClassifier(random_state=156)
scores = cross_val_score(rfr, iris.data, iris.target, scoring='accuracy', cv=3)
print('원본 데이터 교차 검증 개별 정확도:', scores)
print('원본 데이터 평균 정확도:', np.mean(scores))
```

원본 데이터 교차 검증 개별 정확도: [0.98 0.94 0.96]
원본 데이터 평균 정확도: 0.96

2 . PCA로 변환된 데이터 세트에 Random Forest 적용

```
[11] pca_X = irisDF_pca[['pca_component_1', 'pca_component_2']]
scores_pca = cross_val_score(rfr, pca_X, iris.target, scoring='accuracy', cv=3)
print('PCA 변환 데이터 교차 검증 개별 정확도:', scores_pca)
print('PCA 변환 데이터 평균 정확도:', np.mean(scores_pca))
```

PCA 변환 데이터 교차 검증 개별 정확도: [0.88 0.88 0.88]
PCA 변환 데이터 평균 정확도: 0.88

원본 데이터 세트 대비 예측 정확도 하락
→ PCA 변환 차원 개수의 감소
→ 약 8%의 하락

속성 : 4개 → 2개로 감소

⇒ 속성 개수가 50% 감소한 것 고려,
PCA 변환 후에도 원본 데이터 특성
유지한 것!

6.2 PCA(Principal Component Analysis)

[PCA 실습 – 신용카드 고객 데이터 세트]

데이터 세트 DataFrame으로 로딩 (pca_credit_card.xls)

```
[13] # header로 의미 없는 첫 행 제거, 그이로 기존 id 제거
import pandas as pd

df = pd.read_excel('pca_credit_card.xls', header=1, sheet_name='Data').iloc[0:, 1:]
print(df.shape)
df.head(3)
```

(30000, 24)

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default payment next month
0	20000	2	2	1	24	2	2	-1	-1	-2	...	0	0	0	0	689	0	0	0	0	
1	120000	2	2	2	26	-1	2	0	0	0	...	3272	3455	3261	0	1000	1000	1000	0	2000	
2	90000	2	2	2	34	0	0	0	0	0	...	14331	14948	15549	1518	1500	1000	1000	1000	5000	

3 rows × 24 columns

칼럼명 변경 후, 칼럼을 변수로 저장

```
[14] df.rename(columns={'PAY_0': 'PAY_1', 'default payment next month': 'default'}, inplace=True)
y_target = df['default']
X_features = df.drop('default', axis=1)
```

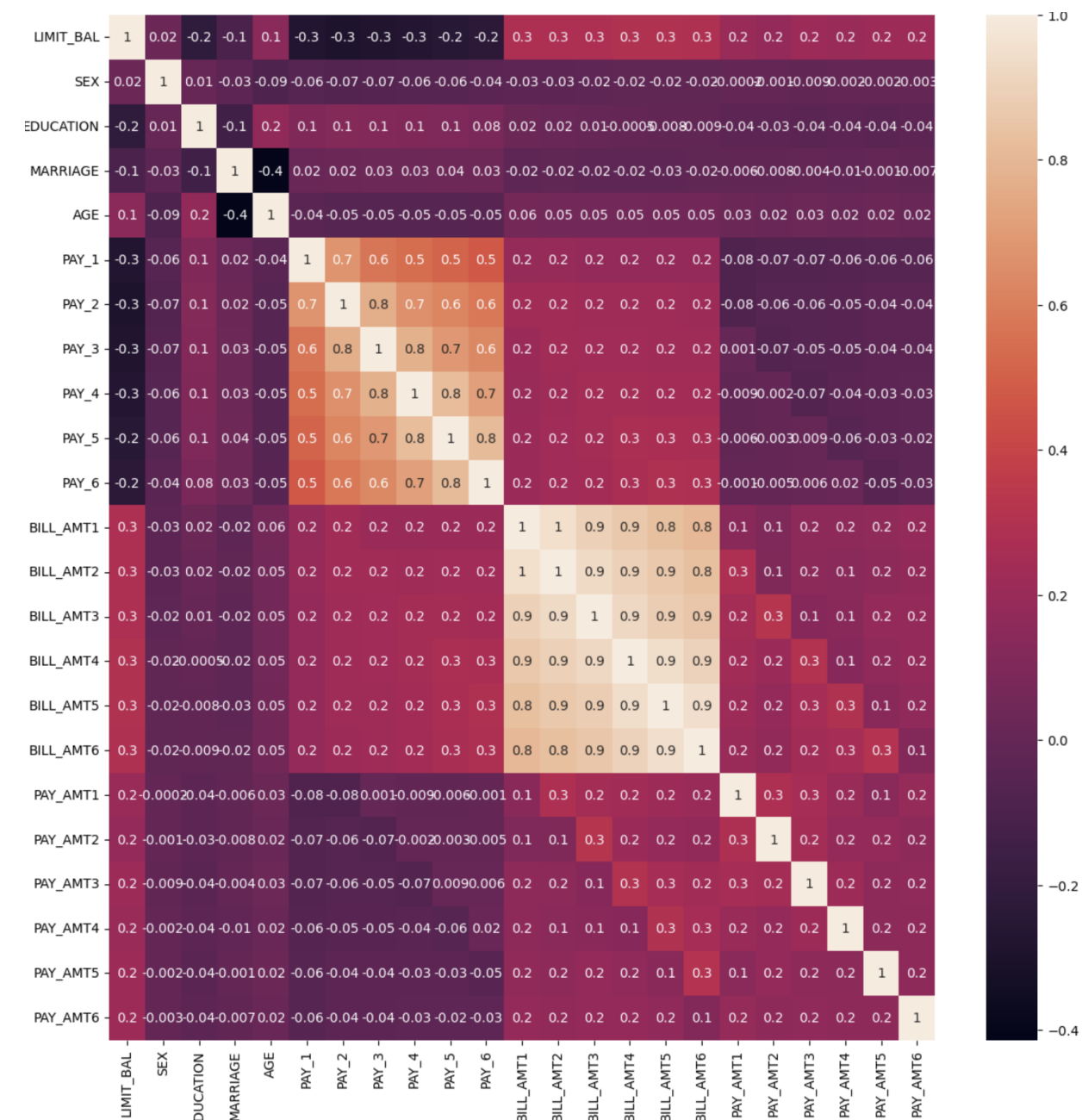
6.2 PCA(Principal Component Analysis)

[PCA 실습 – 신용카드 고객 데이터 세트]

DataFrame 이용해 속성 간 상관도 구하고, heatmap으로 시각화

```
[15] import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

corr = X_features.corr()
plt.figure(figsize=(14, 14))
sns.heatmap(corr, annot=True, fmt='.1g')
```



6.2 PCA(Principal Component Analysis)

[PCA 실습 – 신용카드 고객 데이터 세트]

6개 속성 -> 2개의 component로 PCA 변환, 개별 component 변동성 확인

```
[16] from sklearn.decomposition import PCA
      from sklearn.preprocessing import StandardScaler

      # BILL_AMT1 ~ BILL_AMT6까지 6개의 속성명 생성
      cols_bill = ['BILL_AMT' + str(i) for i in range(1, 7)]
      print('대상 속성명:', cols_bill)

      # 2개의 PCA 속성을 가진 PCA 객체 생성하고, explained_variance_ratio_ 계산을 위해 fit( ) 호출
      scaler = StandardScaler()
      df_cols_scaled = scaler.fit_transform(X_features[cols_bill])
      pca = PCA(n_components=2)
      pca.fit(df_cols_scaled)
      print('PCA Component 별 변동성:', pca.explained_variance_ratio_)
```

```
➡ 대상 속성명: ['BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6']
   PCA Component 별 변동성: [0.9055253 0.0509867 ]
```


6.2 PCA(Principal Component Analysis)

[PCA 실습 – 신용카드 고객 데이터 세트]

{원본 데이터 세트 & PCA 변환한 데이터 세트의 분류 예측 결과 비교}

1. 원본 데이터 세트에 랜덤 포레스트를 이용해 3개의 교차 검증 세트로 분류 예측

```
[17] import numpy as np
      from sklearn.ensemble import RandomForestClassifier
      from sklearn.model_selection import cross_val_score

      rcf = RandomForestClassifier(n_estimators=300, random_state=156)
      scores = cross_val_score(rcf, X_features, y_target, scoring='accuracy', cv=3)

      print('CV=3인 경우의 개별 Fold 세트별 정확도:', scores)
      print('평균 정확도:{0:.4f}'.format(np.mean(scores)))
```

⇒ CV=3인 경우의 개별 Fold 세트별 정확도: [0.8083 0.8196 0.8232]
평균 정확도:0.8170

6.2 PCA(Principal Component Analysis)

[PCA 실습 – 신용카드 고객 데이터 세트]

{원본 데이터 세트 & PCA 변환한 데이터 세트의 분류 예측 결과 비교}

2. PCA 변환한 데이터 세트에 대해서 분류 예측 적용

```
[18] from sklearn.decomposition import PCA
      from sklearn.preprocessing import StandardScaler

      # 원본 데이터 세트에 먼저 StandardScaler 적용
      scaler = StandardScaler()
      df_scaled = scaler.fit_transform(X_features)

      # 6개의 컴포넌트를 가진 PCA 변환을 수행하고 cross_val_score( )로 분류 예측 수행.
      pca = PCA(n_components=6)
      df_pca = pca.fit_transform(df_scaled)
      scores_pca = cross_val_score(rf, df_pca, y_target, scoring='accuracy', cv=3)

      print('CV=3 인 경우의 PCA 변환된 개별 Fold 세트별 정확도:', scores_pca)
      print('PCA 변환 데이터 세트 평균 정확도:{0:.4f}'.format(np.mean(scores_pca)))
```

→ CV=3 인 경우의 PCA 변환된 개별 Fold 세트별 정확도: [0.7912 0.7974 0.802]
PCA 변환 데이터 세트 평균 정확도:0.7969

6.2 PCA(Principal Component Analysis)

[PCA 실습 – 신용카드 고객 데이터 세트]

{원본 데이터 세트 & PCA 변환한 데이터 세트의 분류 예측 결과 비교}

2. PCA 변환한 데이터 세트에 대해서 분류 예측 적용

```
[18] from sklearn.decomposition import PCA
      from sklearn.preprocessing import StandardScaler

      # 원본 데이터 세트에 먼저 StandardScaler 적용
      scaler = StandardScaler()
      df_scaled = scaler.fit_transform(X_features)

      # 6개의 컴포넌트를 가진 PCA 변환을 수행하고 cross_val_score( )로 분류 예측 수행.
      pca = PCA(n_components=6)
      df_pca = pca.fit_transform(df_scaled)
      scores_pca = cross_val_score(rf, df_pca, y_target, scoring='accuracy', cv=3)

      print('CV=3 인 경우의 PCA 변환된 개별 Fold 세트별 정확도:', scores_pca)
      print('PCA 변환 데이터 세트 평균 정확도:{0:.4f}'.format(np.mean(scores_pca)))
```

⇒ CV=3 인 경우의 PCA 변환된 개별 Fold 세트별 정확도: [0.7912 0.7974 0.802]
PCA 변환 데이터 세트 평균 정확도:0.7969

전체 23개의 속성 중 6개의 PCA 컴포넌트
→ 원본 데이터 기반 예측 결과보다
1~2%만 저하!
⇒ PCA의 뛰어난 압축 능력 확인 가능!

* Computer Vision 분야에서 활발한 사용

6.3 LDA (Linear Discriminant Analysis)



6.2 PCA(Principal Component Analysis)

LDA 개요

LDA(Linear Discriminant Analysis) : 선형 판별 분석법
[PCA & LDA 비교]

	PCA	LDA
공통점	입력 데이터 세트를 저차원 공간에 투영해 차원을 축소하기	
차이점	입력 데이터의 결정 값 클래스를 최대한으로 분리할 수 있는 축을 찾음	입력 데이터의 변동성의 가장 큰 축을 찾음

6.2 PCA(Principal Component Analysis)

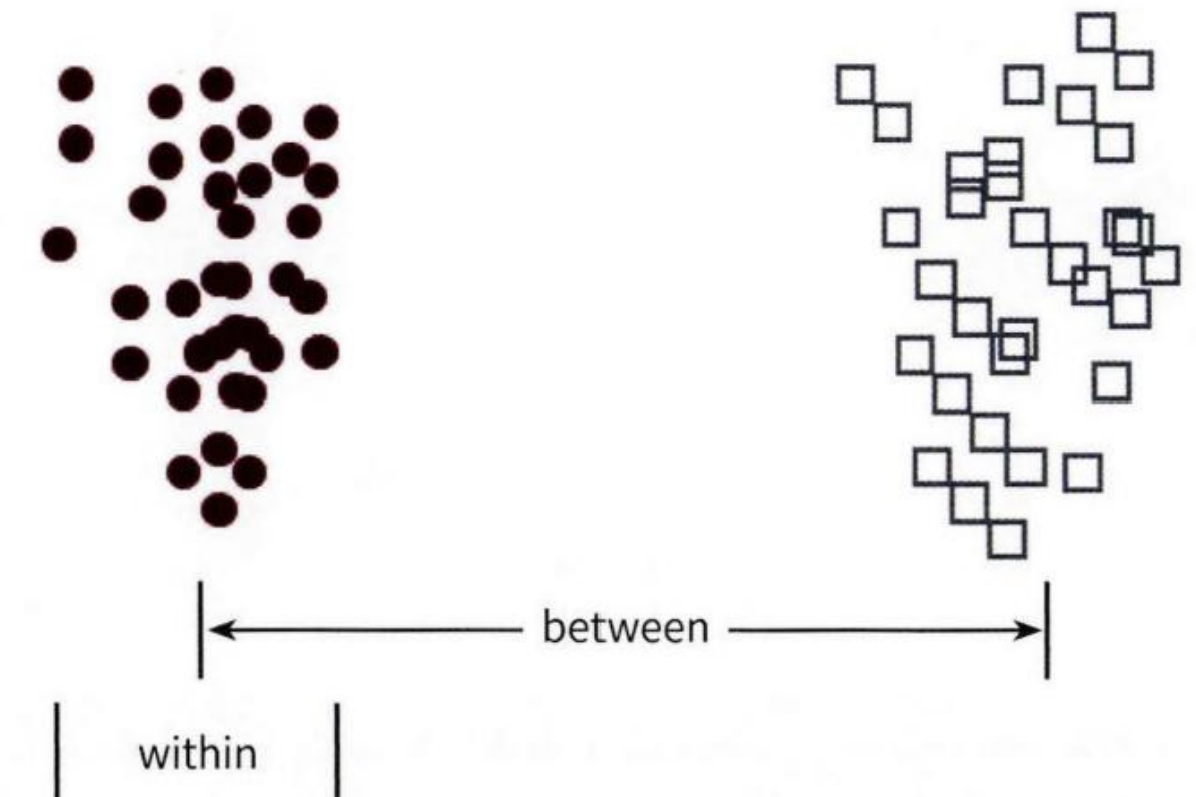
LDA 개요

클래스 분리 최대화하는 축 찾기

→ 클래스 간 분산(between-class scatter)

& 클래스 내부 분산(within-class scatter) 비율 최대화

⇒ 클래스 간 분산 최대한 크게, 클래스 내부 분산 최대한 작게



6.2 PCA(Principal Component Analysis)

LDA 개요

1. 클래스 내부와 클래스 간 분산 행렬 구하기
2. 클래스 내부 분산 행렬을 S_W , 클래스 간 분산 행렬을 $S_B \rightarrow$ 고유벡터로 분해

$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

3. 고유값이 가장 큰 순으로 K개만큼 추출
4. 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

* PCA는 1번 과정에서 공분산 행렬을 이용 !

6.2 PCA(Principal Component Analysis)

붓꽃 데이터 세트에 LDA 적용하기

붓꽃 데이터 세트 로드 & 표준 정규 분포로 스케일링

```
[19] from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
      from sklearn.preprocessing import StandardScaler
      from sklearn.datasets import load_iris

      iris = load_iris()
      iris_scaled = StandardScaler().fit_transform(iris.data)
```

2개의 컴포넌트로 붓꽃 데이터를 LDA 변환

```
[22] lda = LinearDiscriminantAnalysis(n_components=2)
      lda.fit(iris_scaled, iris.target)
      iris_lda = lda.transform(iris_scaled)
      print(iris_lda.shape)
```

→ (150, 2)

6.2 PCA(Principal Component Analysis)

붓꽃 데이터 세트에 LDA 적용하기

LDA 변환된 입력 데이터 값을 2차원 평면에 품종별로 표현

```
[24] import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

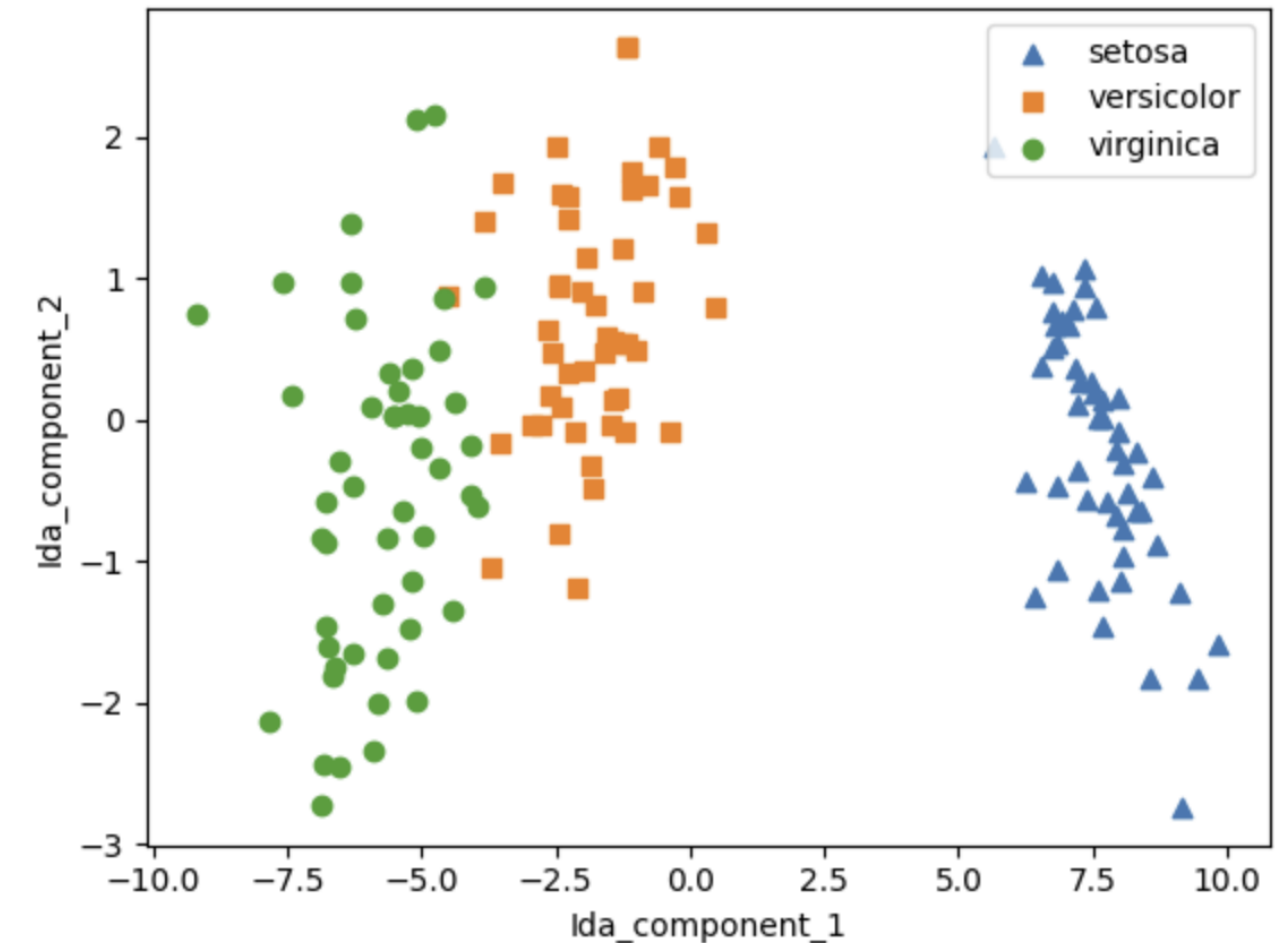
lda_columns=['lda_component_1', 'lda_component_2']
irisDF_lda = pd.DataFrame(iris_lda, columns=lda_columns)
irisDF_lda['target']=iris.target

#setosa는 세모, versicolor는 네모, virginica는 동그라미로 표현
markers = ['^', 's', 'o']

#setosa의 target 값은 0, versicolor는 1, virginica는 2. 각 target별로 다른 모양으로 산점도로 표시
for i, marker in enumerate(markers):
    x_axis_data = irisDF_lda [irisDF_lda ['target']==i] ['lda_component_1']
    y_axis_data = irisDF_lda [irisDF_lda ['target']==i] ['lda_component_2']

    plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

plt.legend(loc='upper right')
plt.xlabel('lda_component_1')
plt.ylabel('lda_component_2')
plt.show()
```



PCA로 변환된 데이터와 좌우 대칭으로
닮아 있음

6.4 SVD



#01 SVD (Singular Value Decomposition)

PCA – 주성분 분석 -> 가장 높은 **분산**을 가지는 데이터의 축을 찾아 차원 축소

공분산 $C = P \Sigma P^T = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$ **고유벡터**

SVD = Singular Value Decomposition = **특이값 분해**

$$\begin{matrix} & m \times n \\ & \text{특이값} \\ \begin{matrix} m \times n \\ A \end{matrix} & = & \begin{matrix} m \times m \\ U \end{matrix} & \begin{matrix} \Sigma \end{matrix} & \begin{matrix} n \times n \\ V^T \end{matrix} \end{matrix}$$

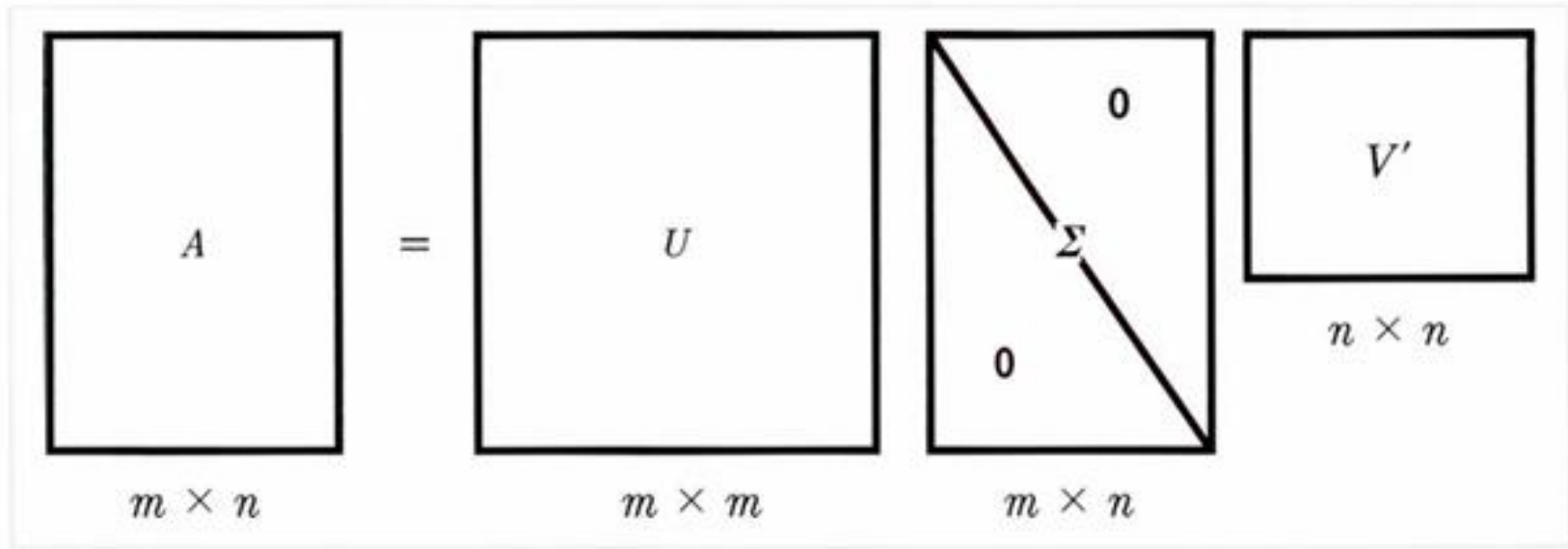
- 임의의 $m \times n$ 행렬 A 를 특이값을 이용해 이와 같이 분해 가능
- 원리) 특이벡터를 활용한 선형 변환

$$AV = U\Sigma$$

V : 서로 직교하는 벡터 $\vec{x}, \vec{y}$ 로 이루어진 특이행렬
 U : 특이행렬의 단위행렬 (크기가 1)
 Σ : 특이값

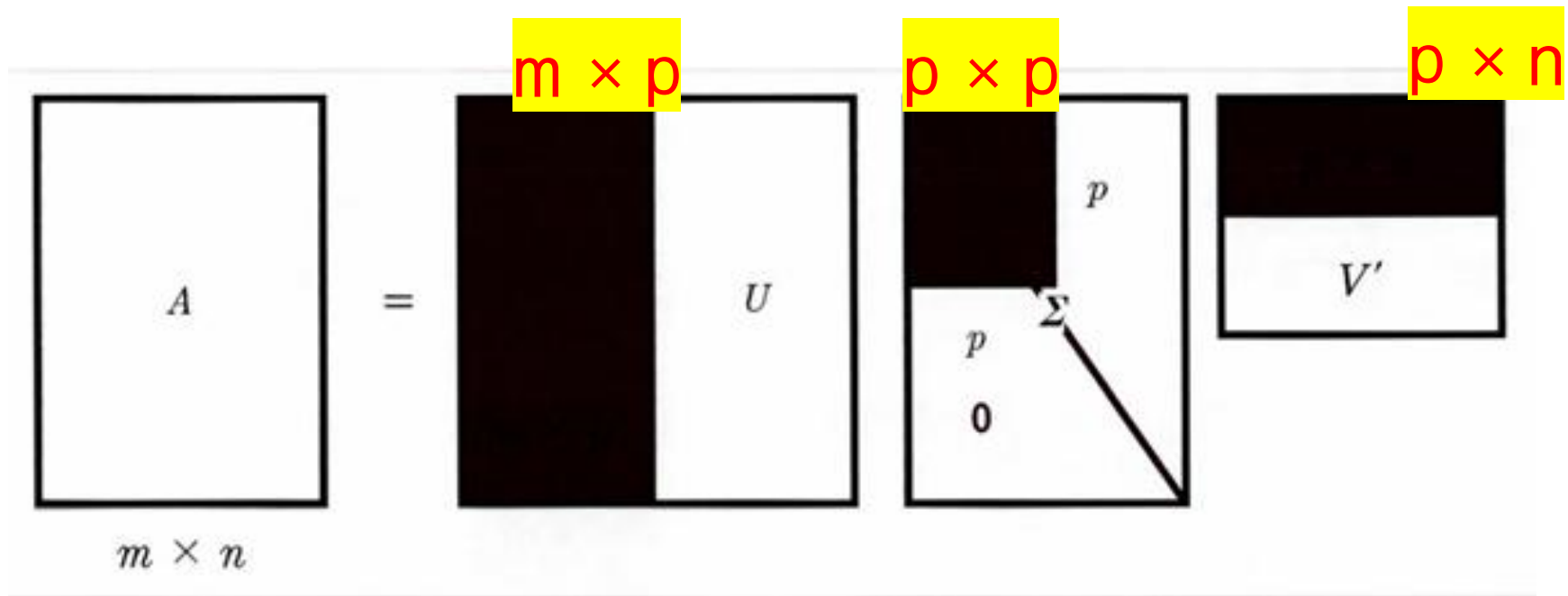
#01 SVD를 이용한 차원 축소

$$A = U \Sigma V^T$$



차원 축소

1. 특이값이 0인 부분 제거
2. 제거된 Σ 에 대응하는 U , V 원소 함께 제거
=> 특이값을 p 개만 이용해
부분 복원 == 차원 축소



Truncated SVD

특이값 중 상위 일부 데이터만 추출해 분해하는 방식

#02 SVD 연산 & 분해 예제

넘파이의 SVD 모듈 `numpy.linalg.svd`
4X4 넘파이 행렬 `a` 생성

```
# 넘파이의 svd 모듈 임포트
import numpy as np
from numpy.linalg import svd

# 4x4 랜덤 행렬 A
np.random.seed(121)
a = np.random.rand(4,4)
print(np.round(a,3))
```

4X4 행렬 `a`

```
[[0.111 0.211 0.233 0.152]
 [0.83  0.408 0.556 0.746]
 [0.248 0.969 0.953 0.49 ]
 [0.018 0.858 0.405 0.622]]
```

`a` 행렬에 SVD 적용
→ `U`, `Sigma( $\Sigma$ )`, `VT` 반환

- `U`, `Vt`: 4X4
- `Sigma( $\Sigma$ )` → (4,)
: 0이 아닌 값만 1차원 행렬로 표현

```
U, Sigma, Vt = svd(a)
print(U.shape, Sigma.shape, Vt.shape)
print()
print('U matrix:\n', np.round(U,3))
print()
print('Sigma Value:\n', np.round(Sigma, 3))
print()
print('V transpose matrix:\n', np.round(Vt,3))
print()
```

(4, 4) (4,) (4, 4)

U matrix:

```
[[-0.169  0.001 -0.128 -0.977]
 [-0.541 -0.828  0.126  0.076]
 [-0.662  0.355 -0.63   0.197]
 [-0.491  0.434  0.756 -0.014]]
```

Sigma Value:

```
[2.153 0.725 0.356 0.007]
```

V transpose matrix:

```
[[-0.298 -0.612 -0.543 -0.492]
 [-0.815  0.522  0.075 -0.239]
 [-0.148  0.174 -0.714  0.662]
 [ 0.474  0.568 -0.436 -0.513]]
```

#02 SVD 연산 & 분해 예제

분해된 U, Sigma, Vt -> 원본 행렬 복원

```
# Sigma를 다시 0을 포함한 대칭 행렬로 변환
Sigma_mat = np.diag(Sigma)
a_ = np.dot(np.dot(U, Sigma_mat), Vt)
print(np.round(a_, 3))
```

```
[[0.111 0.211 0.233 0.152]
 [0.83  0.408 0.556 0.746]
 [0.248 0.969 0.953 0.49 ]
 [0.018 0.858 0.405 0.622]]
```

4X4 행렬 a

```
[[0.111 0.211 0.233 0.152]
 [0.83  0.408 0.556 0.746]
 [0.248 0.969 0.953 0.49 ]
 [0.018 0.858 0.405 0.622]]
```

* Sigma: 0을 포함한 대칭행렬로 변환 후 내적 수행할 것

#02 SVD 연산 & 분해 예제

로우 간 의존성이 있는 데이터 세트

```
a[2] = a[0] + a[1]
a[3] = a[0]
print(np.round(a, 3))
```

```
[[0.111 0.211 0.233 0.152]
 [0.83  0.408 0.556 0.746]
 [0.942 0.619 0.789 0.897]
 [0.111 0.211 0.233 0.152]]
```

- 3번째 로우 = 1번째 로우 + 2번째 로우
- 4번째 로우 = 1번째 로우



```
(4, 4) (4,) (4, 4)
```

U matrix:

```
[[-0.156  0.613  0.635 -0.443]
 [-0.607 -0.481  0.542  0.326]
 [-0.763  0.132 -0.542 -0.326]
 [-0.156  0.613 -0.093  0.769]]
```

Sigma Value:

```
[2.151 0.24  0.   0.   ]
```

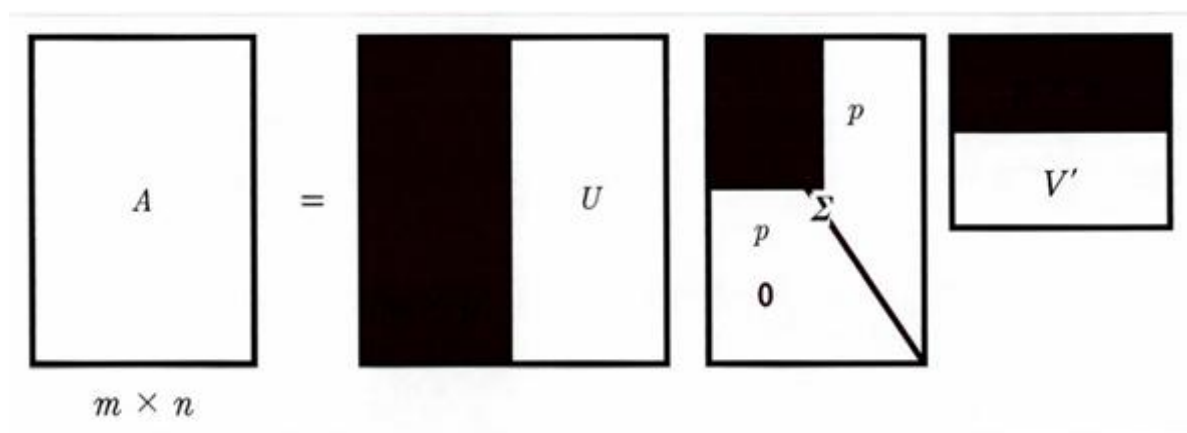
V transpose matrix:

```
[[-0.585 -0.365 -0.471 -0.551]
 [-0.575  0.599  0.51  -0.223]
 [-0.57  -0.173 -0.092  0.798]
 [ 0.047  0.691 -0.714  0.101]]
```

- Sigma 값 중 2개 0이 됨
= 선형 독립인 로우 벡터의 수(Rank)가 2

#02 SVD 연산 & 분해 예제

Sigma가 0인 데이터 제외하고 복원



```
# Sigma를 다시 0을 포함한 대칭 행렬로 변환
Sigma_mat = np.diag(Sigma)
a_ = np.dot(np.dot(U, Sigma_mat), Vt)
print(np.round(a_, 3))
```

```
[[0.111 0.211 0.233 0.152]
 [0.83  0.408 0.556 0.746]
 [0.248 0.969 0.953 0.49 ]
 [0.018 0.858 0.405 0.622]]
```

```
# U 행렬의 경우 Sigma와 내적 수행 -> Sigma 앞 2행에 대응되는 앞 2열만 추출
U_ = U[:, :2]
Sigma_ = np.diag(Sigma[:2])
# V 전치 행렬의 경우는 앞 2행만 추출
Vt_ = Vt[:2]
print(U_.shape, Sigma_.shape, Vt_.shape)
# U, Sigma, Vt의 내적을 수행하며, 다시 원본 행렬 복원
a_ = np.dot(np.dot(U_, Sigma_), Vt_)
print(np.round(a_, 3))
```

```
(4, 2) (2, 2) (2, 4)
[[0.111 0.211 0.233 0.152]
 [0.83  0.408 0.556 0.746]
 [0.942 0.619 0.789 0.897]
 [0.111 0.211 0.233 0.152]]
```

#03 Truncated SVD로 행렬 분해

사이파이 Truncated SVD로 실습 – `scipy.linalg.svd`

- 특이값 4개로 Truncated SVD
- 근사적 복원

```
import numpy as np
from scipy.sparse.linalg import svds
from scipy.linalg import svd

# 원본 행렬 출력하고 SVD를 적용할 경우 U, Sigma, Vt의 차원 확인
np.random.seed(121)
matrix = np.random.random((6,6))
print('원본 행렬:\n', matrix)
U, Sigma, Vt = svd(matrix, full_matrices=False)
print('\n분해 행렬 차원:', U.shape, Sigma.shape, Vt.shape)
print('\nSigma값 행렬:', Sigma)

# Truncated SVD로 Sigma 행렬의 특이값을 4개로 하여 Truncated SVD 수행
num_components = 4
U_tr, Sigma_tr, Vt_tr = svds(matrix, k=num_components)
print('\nTruncated SVD 분해 행렬 차원:', U_tr.shape, Sigma_tr.shape, Vt_tr.shape)
print('\nTruncated SVD Sigma값 행렬:', Sigma_tr)
matrix_tr = np.dot(np.dot(U_tr, np.diag(Sigma_tr)), Vt_tr) # TruncatedSVD의 결과

print('\nTruncated SVD로 분해 후 복원 행렬\n', matrix_tr)
```

원본 행렬:

```
[[0.11133083 0.21076757 0.23296249 0.15194456 0.83017814 0.40791941]
 [0.5557906  0.74552394 0.24849976 0.9686594  0.95268418 0.48984885]
 [0.01829731 0.85760612 0.40493829 0.62247394 0.29537149 0.92958852]
 [0.4056155  0.56730065 0.24575605 0.22573721 0.03827786 0.58098021]
 [0.82925331 0.77326256 0.94693849 0.73632338 0.67328275 0.74517176]
 [0.51161442 0.46920965 0.6439515  0.82081228 0.14548493 0.01806415]]
```

분해 행렬 차원: (6, 6) (6,) (6, 6)

Sigma값 행렬: [3.2535007 0.88116505 0.83865238 0.55463089 0.35834824 0.0349925]

Truncated SVD 분해 행렬 차원: (6, 4) (4,) (4, 6)

Truncated SVD Sigma값 행렬: [0.55463089 0.83865238 0.88116505 3.2535007]

Truncated SVD로 분해 후 복원 행렬"

```
[[0.19222941 0.21792946 0.15951023 0.14084013 0.81641405 0.42533093]
 [0.44874275 0.72204422 0.34594106 0.99148577 0.96866325 0.4754868 ]
 [0.12656662 0.88860729 0.30625735 0.59517439 0.28036734 0.93961948]
 [0.23989012 0.51026588 0.39697353 0.27308905 0.05971563 0.57156395]
 [0.83806144 0.78847467 0.93868685 0.72673231 0.6740867  0.73812389]
 [0.59726589 0.47953891 0.56613544 0.80746028 0.13135039 0.03479656]]
```


#03 Truncated SVD로 행렬 분해

사이킷런 TruncatedSVD 클래스를 이용한 변환

- U, Sigma, Vt 행렬을 반환하지는 않음
- fit(), transform() 호출해서 차원 축소 수행
 - > 원본 데이터를 U * Sigma 행렬에 선형 변환해 생성

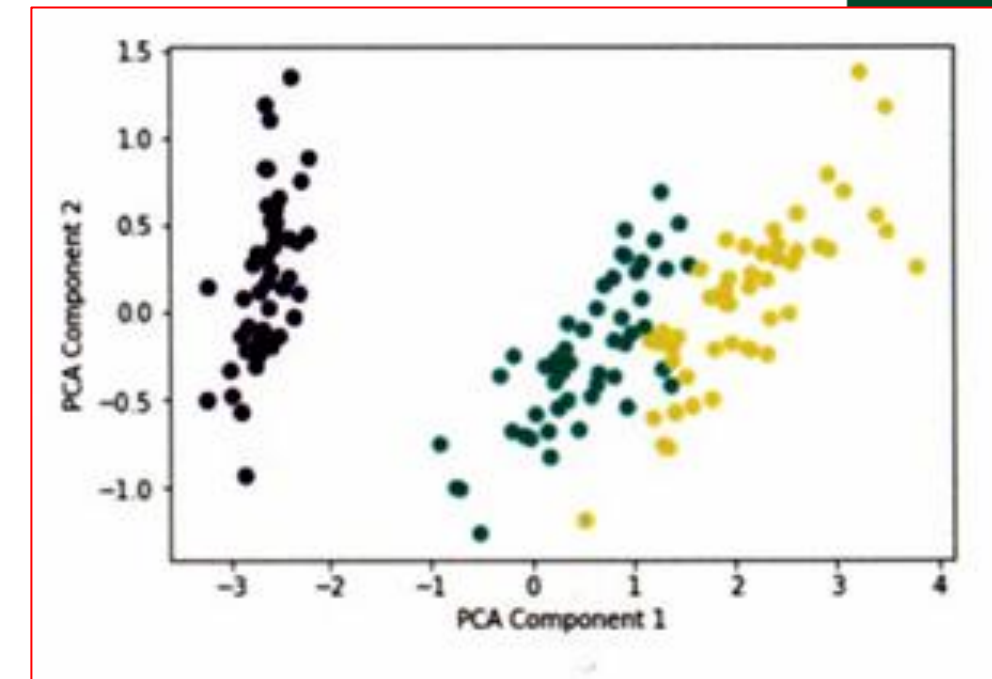
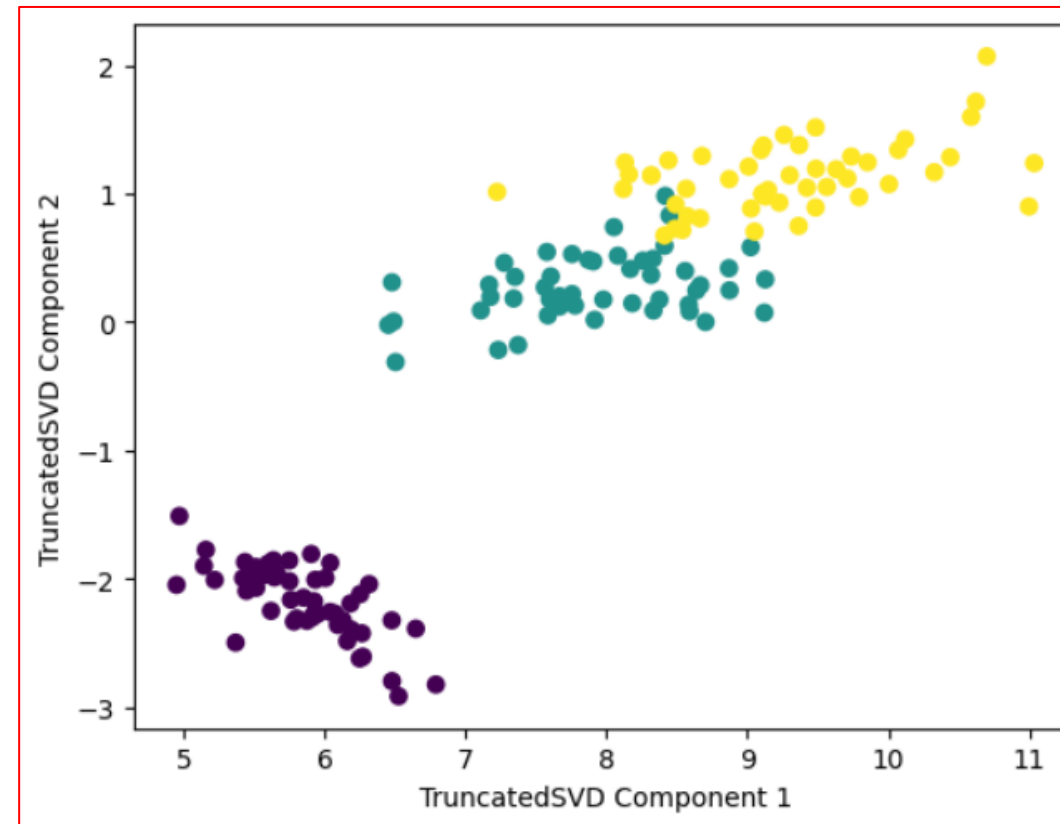
$$A = U \Sigma V^T$$

$$AV \neq U\Sigma$$

```
from sklearn.decomposition import TruncatedSVD, PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline

iris = load_iris()
iris_fts = iris.data
# 2개의 주요 component로 TruncatedSVD 변환
tsvd = TruncatedSVD(n_components=2)
tsvd.fit(iris_fts)
iris_tsvd = tsvd.transform(iris_fts)

# Scatter plot 2차원으로 TruncatedSVD 변환 된 데이터 표현
plt.scatter(x=iris_tsvd[:,0], y= iris_tsvd[:,1], c= iris.target)
plt.xlabel('TruncatedSVD Component 1')
plt.ylabel('TruncatedSVD Component 2')
```



TruncatedSVD로 변환된 데이터 세트

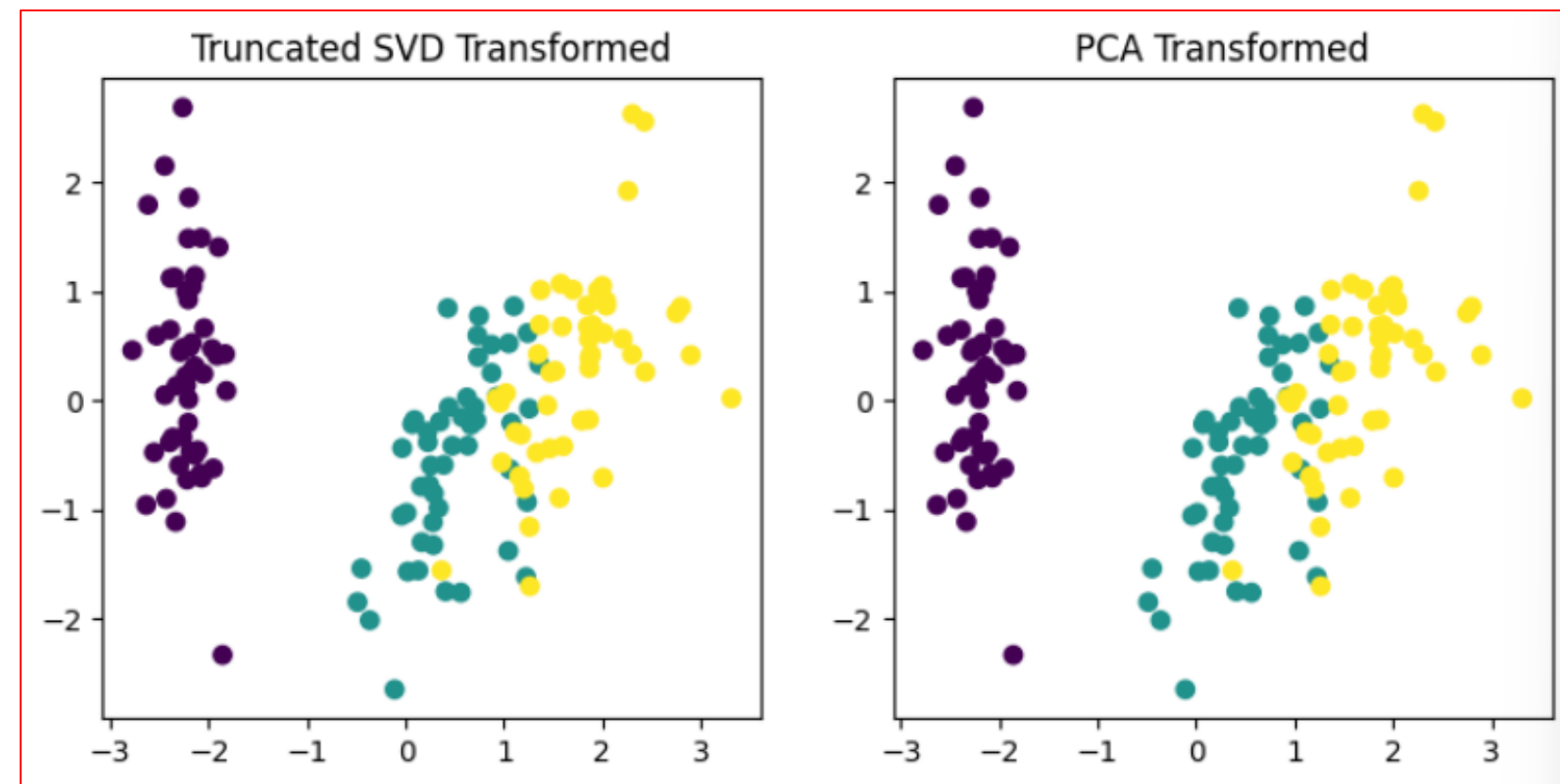
#03 Truncated SVD로 행렬 분해

사이킷런 TruncatedSVD 클래스를 이용한 변환

- 붓꽃 데이터를 **스케일링**으로 변환한 뒤 TruncatedSVD, PCA 클래스 변환 -> 거의 동일한 결과

```
# 스케일링된 데이터를 기반으로 TruncatedSVD 변환 수행
tsvd = TruncatedSVD(n_components=2)
tsvd.fit(iris_scaled)
iris_tsvd = tsvd.transform(iris_scaled)
```

```
# 스케일링된 데이터를 기반으로 PCA 변환 수행
pca = PCA(n_components=2)
pca.fit(iris_scaled)
iris_pca = pca.transform(iris_scaled)
```



두 변환 행렬 값과 원본 속성별 컴포넌트 비율값

$\text{mean}(\text{iris_pca} - \text{iris_tsvd})$

$\text{mean}(\text{pca.components} - \text{tsvd.components})$

-> 모두 0에 가까움 => 2개의 변환이 서로 동일

```
print((iris_pca - iris_tsvd).mean())
print((pca.components_ - tsvd.components_).mean())
```

```
2.33787526375077e-15
-9.020562075079397e-17
```

6.5 NMF



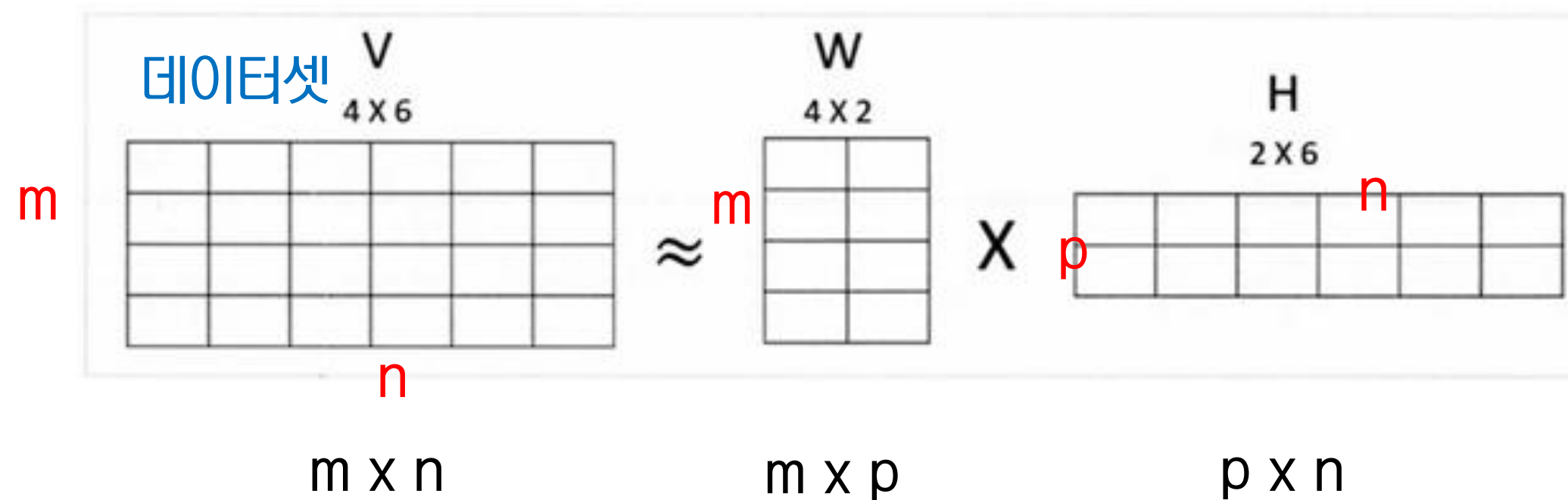
#01 NMF (Non-negative Matrix Factorization)

= 비음수 행렬 분해

하나의 객체정보 -> 음수를 포함하지 않은 두 개의 부분 정보로 인수분해 -> 의미 있는 특징 추출

- 낮은 랭크를 통한 행렬 근사(Low-Rank Approximation) 방식의 변형
(Truncated SVD도 포함)

NMF의 목적: 공통 특성만을 가지고 정보를 줄이는 것



V: 데이터셋 행렬 – 행: sample, 열: feature

p: 축소할 차원의 수

W: Latent Representation 행렬

- 특성이 얼마나 적합한지를 판단하는 가중치

H: Latent Features 행렬

- 각 특성의 중요도

-> V가 W와 H로 근사해 분해됨

행렬 W, H에 초기값을 무작위로 설정

→ 행렬 V와 $W \times H$ 간의 거리를 최소화하는 방향으로 값 갱신 (근사값의 정확도 ↑)

#01 NMF (Non-negative Matrix Factorization)

중배 갱신 규칙 (multiplicative update rules)

1. 4개의 갱신 행렬 W , H , D , N 생성

2. 행렬 W 갱신

- 행렬 W 의 모든 값을 (3)의 값과 곱하고 (4)의 값으로 나누기

3. 행렬 H 갱신

- 행렬 H 의 모든 값을 (5)와 곱하고 (6)의 값으로 나누기

4. 1~3을 행렬 R (데이터셋 행렬)와 행렬 $W \times H$ 의 차이가 0이 될 때까지 반복

$$WN = R \times W^T \quad (3)$$

$$WD = H \times W \times W^T \quad (4)$$

$$HN = H^T \times R \quad (5)$$

$$HD = H^T \times H \times W \quad (6)$$

PCA: 공분산 행렬의 고유 벡터는 항상 직교 → 데이터셋의 실제 데이터 구조를 잘 반영하지 X

NMF: 대량의 정보를 의미 특징 & 의미 변수로 나누어 효율적으로 표현 가능

#02 NMF 변환 실습

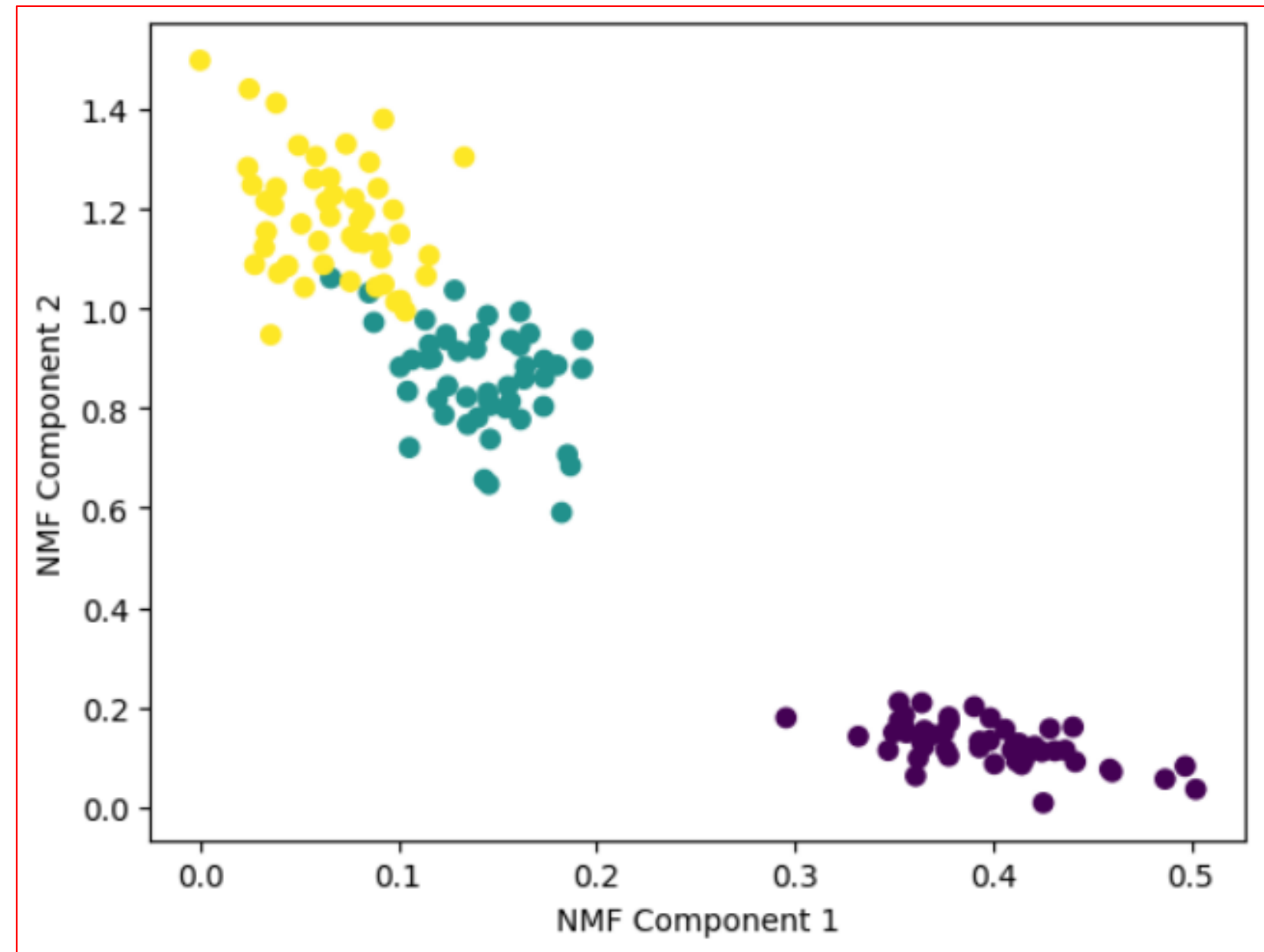
사이킷런의 NMF 클래스 이용

- 붓꽃 데이터를 NMF로 2개의 컴포넌트로 변환 후 시각화

```
# NMF
from sklearn.decomposition import NMF
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
%matplotlib inline

iris = load_iris()
iris_ftrs = iris.data
nmf = NMF(n_components=2)
nmf.fit(iris_ftrs)
iris_nmf = nmf.transform(iris_ftrs)
plt.scatter(x=iris_nmf[:,0], y=iris_nmf[:,1], c=iris.target)
plt.xlabel('NMF Component 1')
plt.ylabel('NMF Component 2')

plt.show()
```



차원축소 기법-보충



랜덤 투영(random projection)

- 랜덤한 선형 투영을 사용해, 데이터를 저차원 공간으로 투영
 - 무작위로 설계된 선형 변환도 고차원 공간에서의 유사도(거리)를 어느정도 유지할 수 있다는 이론적 보장에 기반함
- 대규모 데이터 처리에 효과적이며 데이터 간 유사도(거리)를 잘 보존함
- 품질은 데이터 개수(샘플 수)와 축소할 차원 수에 의해 결정되며, 원래 데이터의 차원 수와는 크게 관계가 없음

다차원 스케일링(MDS)

- 샘플 간 거리(유사도)를 보존하면서 고차원 데이터를 저차원 공간에 배치(차원을 축소)함
- 거리 보존이 핵심 목적이며, PCA처럼 분산 보존은 수행하지 않음
- 거리 구조를 잘 반영해, 시각화에 탁월하며 거리 행렬만 있으면 적용이 가능

Isomap

- 각 샘플을 가장 가까운 이웃과 연결하는 방식으로 그래프를 만들고, 샘플 간의 지오데식 거리(곡면 위의 최단 경로)를 유지하면서 차원을 축소
- MDS의 확장 개념: (MDS는 유클리디안 거리, Isomap은 지오데식 거리를 사용)
- 비선형 구조의 보존에 강하며, 곡면 위 데이터 표현이

t-SNE

- 비슷한 샘플은 가까이, 비슷하지 않은 샘플은 멀리 떨어지도록 차원 축소
- 데이터 간 국소적인 구조(이웃 관계)를 보존
- 군집이 잘 구분되며 직관적 시각화가 가능

선형 판별 분석(LDA)

- 분류 알고리즘이지만, 차원 축소 기법으로도 사용
- 훈련 과정에서 클래스 사이를 가장 잘 구분(클래스 간의 분산은 최대화, 클래스 내부의 분산은 최소화)하는 축을 학습
- 축은 데이터가 투영되는 초평면을 정의, 그 축에 데이터를 투영하여 차원을 줄임

여러가지 기법을 사용해 스위스 롤을 2D로 축소하기

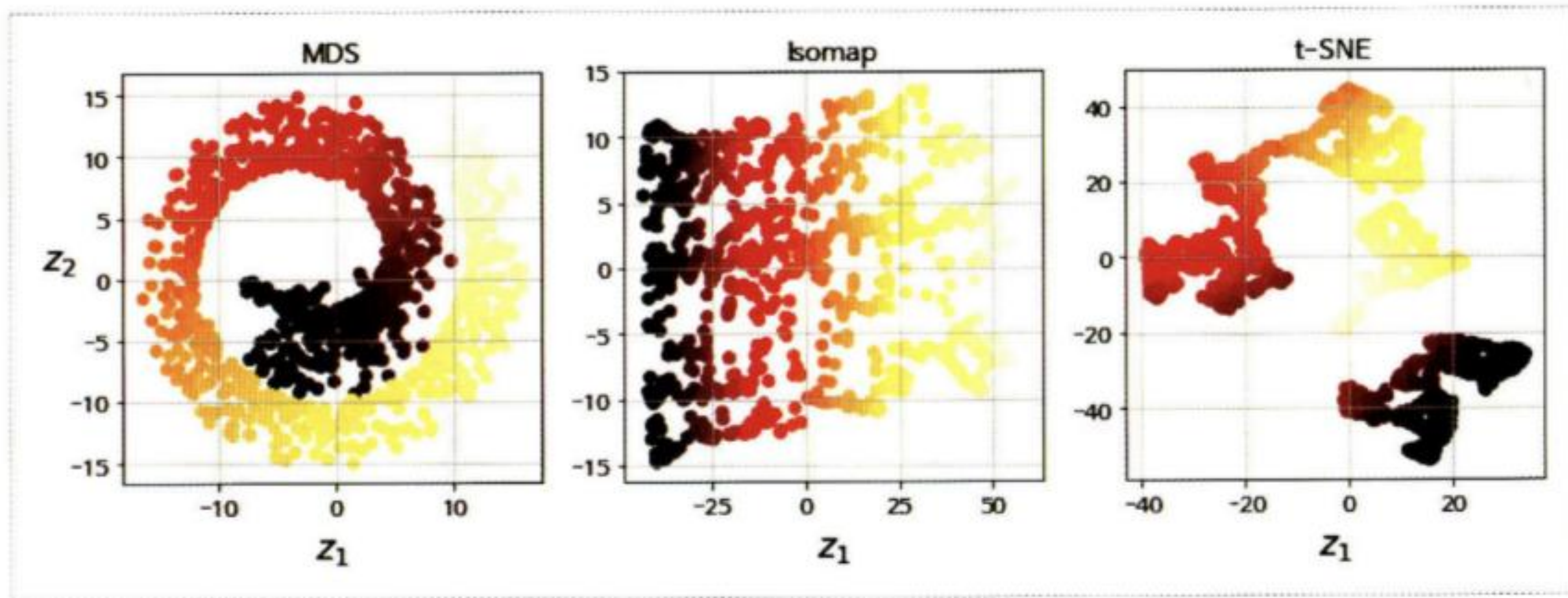


그림 8-13 여러 가지 기법을 사용해 스위스 롤을 2D로 축소하기

Dimensionality Reduction for Beginners



Dimensionality Reduction for Beginners

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from keras.datasets import mnist
```

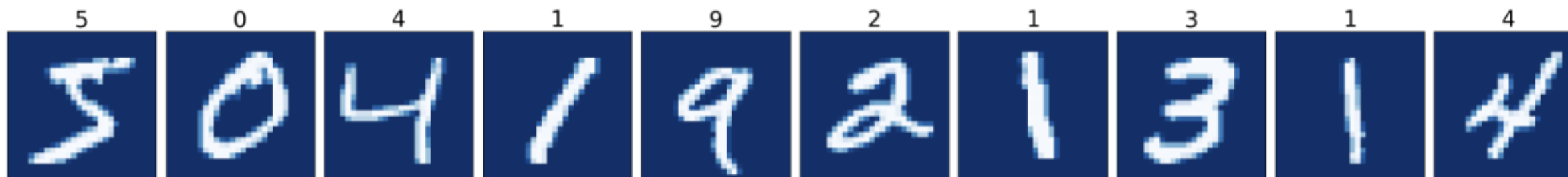
```
from sklearn.decomposition import PCA #Decomposition of multivariate dataset
from sklearn.decomposition import TruncatedSVD #singular value decomposition, similar to PCA, but differs in that the matrix X does not need to be centered
from sklearn.decomposition import NMF #assumes that the data non-negative, only difference from pca is representation of a vector is obtained in an additive fashion
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis #finding linear combinations of features, brings multi to single dimensional and finds class separators
from sklearn.manifold import TSNE #Embedding high-dimensional data for visualization in a low-dimensional space of two or three dimensions
import umap # reduces the dimensions to visualize, patternize and cluster
from umap import UMAP
import umap.plot
import plotly
import plotly.express as px
```

```
(train_x, train_y), (test_x, test_y) = mnist.load_data()
```

Dimensionality Reduction for Beginners

Tranining Data

```
:  
fig, axes = plt.subplots(1, 10, figsize=(25, 4))  
for idx, ax in enumerate(axes):  
    ax.imshow(train_x[idx], cmap='Blues_r')  
    ax.set_title(str(train_y[idx]), fontsize=25)  
    ax.set_xticks([])  
    ax.set_yticks([])  
plt.tight_layout()
```

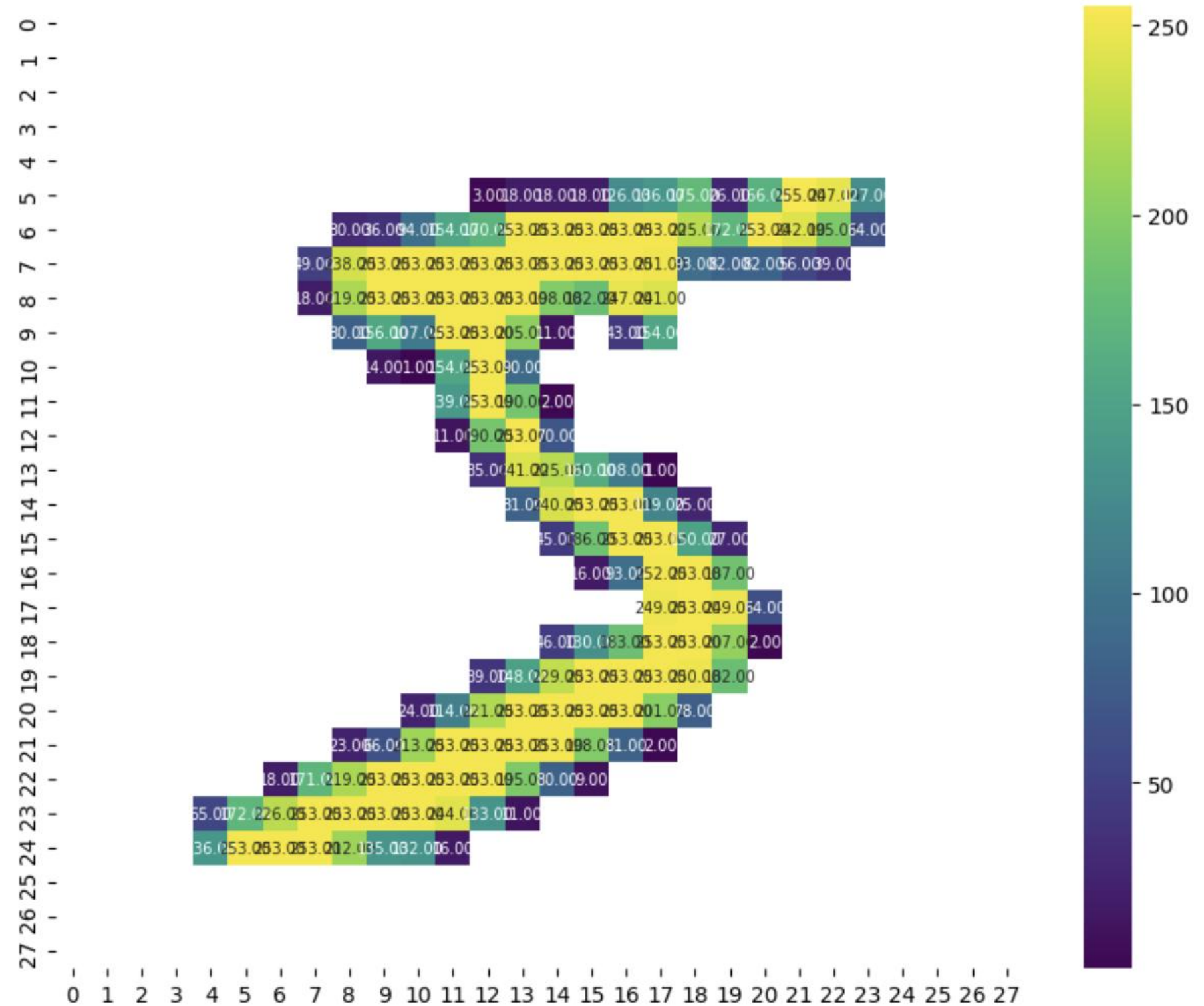


Dimensionality Reduction for Beginners

```
img = train_x[0]

if len(img.shape) > 2:
    img = img.mean(axis=2)
mask = img == 0
fig, ax = plt.subplots(figsize=(10, 8))

sns.heatmap(img, annot=True, fmt='.2f', cmap='viridis',
            mask=mask, cbar=True, ax=ax,
            annot_kws={'size': 7})
```



Dimensionality Reduction for Beginners

```
train_x = train_x.reshape(train_x.shape[0], -1)
print(train_x.shape)

sample_size = 5000
train_x = pd.DataFrame(train_x[:sample_size, :])
train_y = train_y[:sample_size]
```

(60000, 784)

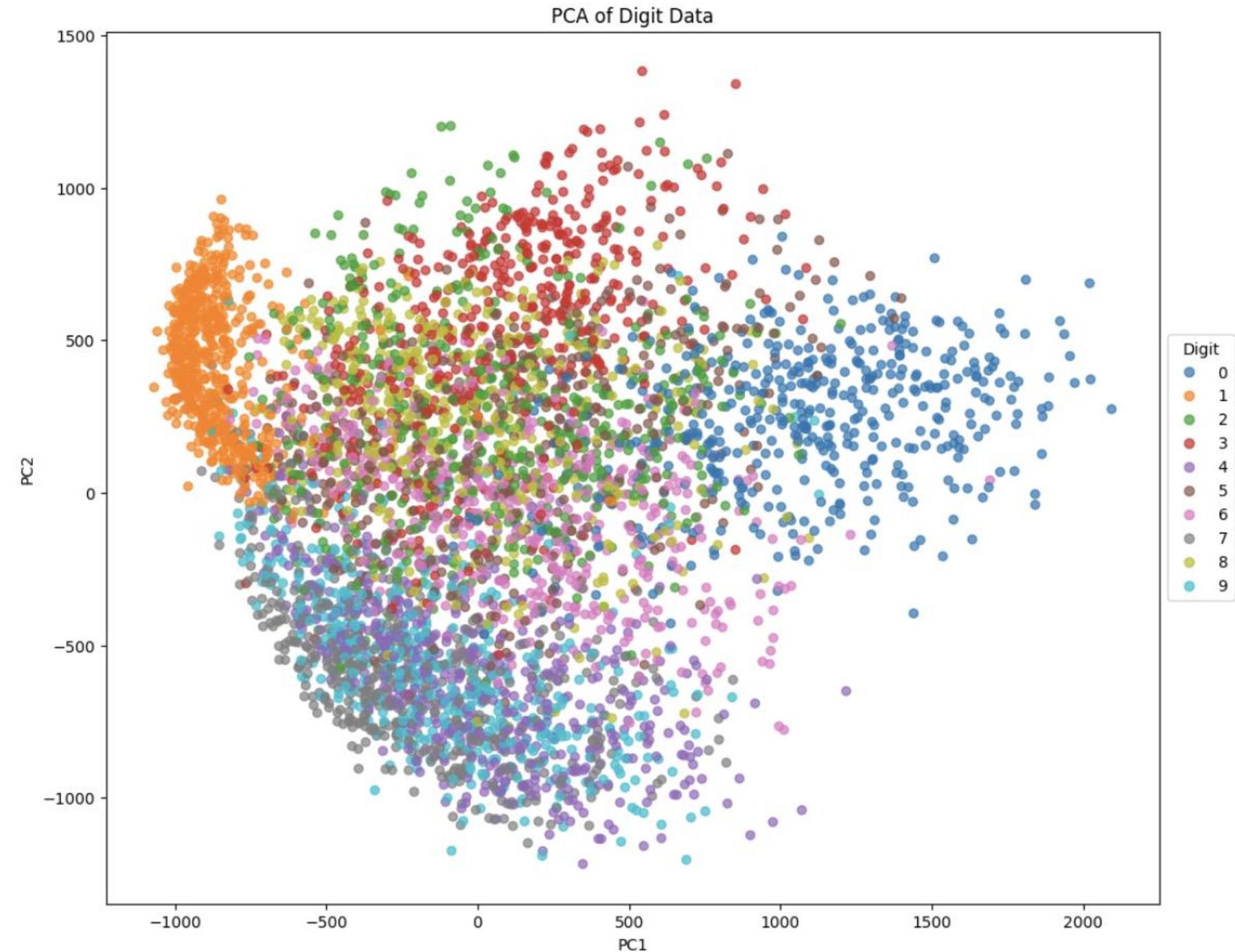
```
#Markers for vizualization
markers = ['o', 'v', '^', '<', '>', '8', 's', 'P', '*', 'X']
```


Dimensionality Reduction for Beginners

```
fig, ax = plt.subplots(figsize=(12, 10))
x_pca = PCA(n_components=2).fit_transform(train_x)

scatter = ax.scatter(x_pca[:, 0], x_pca[:, 1], c=train_y, cmap='tab10',
                    alpha=0.7, s=30)

legend = ax.legend(*scatter.legend_elements(), title="Digit", loc="center left",
                  bbox_to_anchor=(1, 0.5))
ax.set(xlabel='PC1', ylabel='PC2', title='PCA of Digit Data')
```

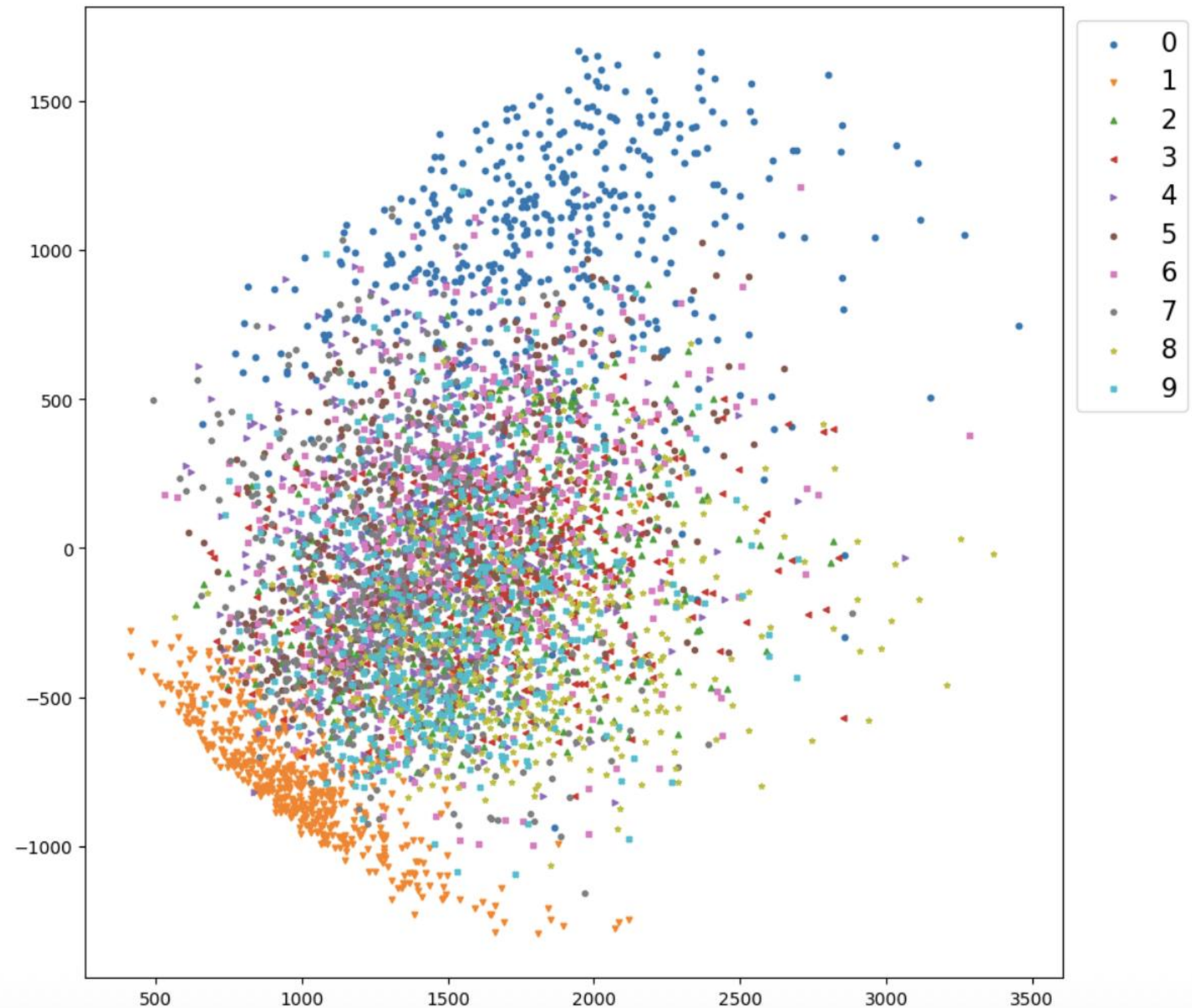


Dimensionality Reduction for Beginners

```
tsvd = TruncatedSVD()
x_tsvd = tsvd.fit_transform(train_x)

plt.figure(figsize=(10, 10))
for i, marker in enumerate(markers):
    plt.scatter(*x_tsvd[train_y == i, :2].T, label=i, s=10, alpha=
1, marker=marker)

plt.legend(bbox_to_anchor=(1.00, 1), loc='upper left', fontsize=15)
plt.show()
```

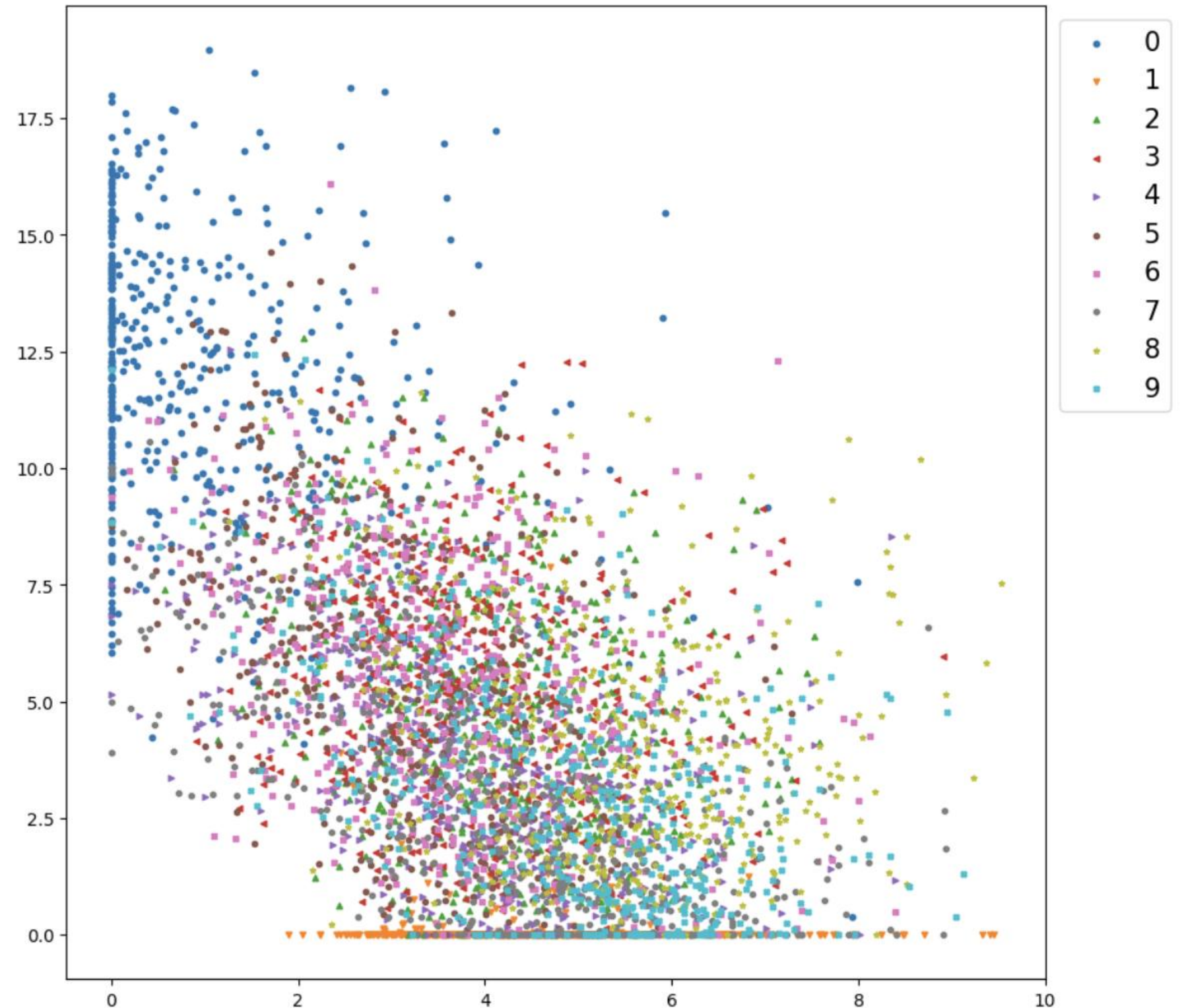


Dimensionality Reduction for Beginners

```
nmf = NMF(n_components=2, init='random', random_state=0)
x_nmf = nmf.fit_transform(train_x)

plt.figure(figsize=(10, 10))
for i, marker in enumerate(markers):
    plt.scatter(*x_nmf[train_y == i, :2].T, label=i, s=10, alpha=1,
                marker=marker)

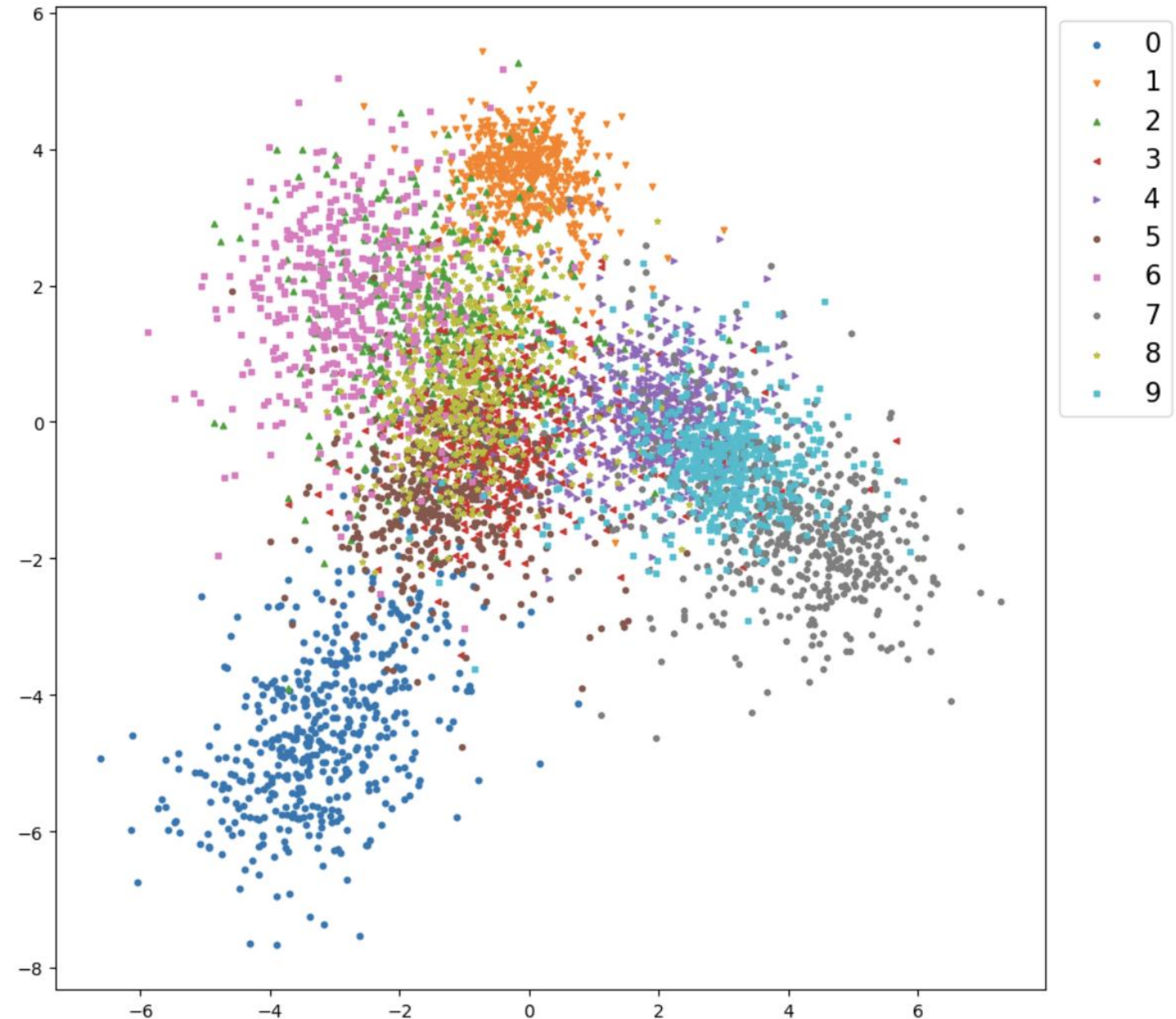
plt.legend(bbox_to_anchor=(1.00, 1), loc='upper left', fontsize=15)
plt.show()
```



Dimensionality Reduction for Beginners

```
lda = LinearDiscriminantAnalysis(n_components=2)
x_lda = lda.fit_transform(train_x, train_y)

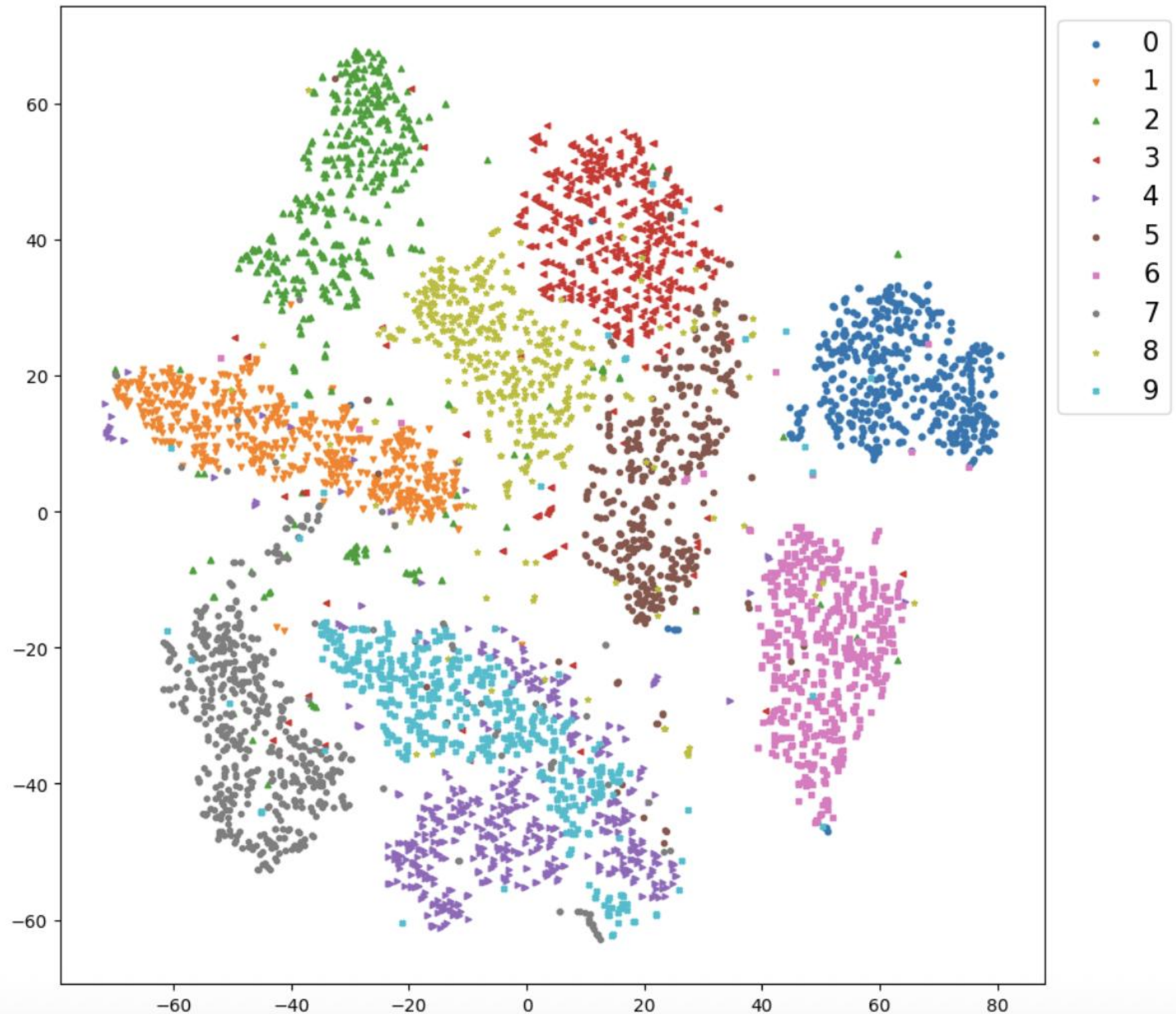
plt.figure(figsize=(10,10))
for i, marker in enumerate(markers):
    plt.scatter(*x_lda[train_y == i].T, label=i, s=10, alpha=1, marker=marker)
plt.legend(bbox_to_anchor=(1.00, 1), loc='upper left', fontsize=15)
```



Dimensionality Reduction for Beginners

```
tsne = TSNE(n_components=2)
x_tsne = tsne.fit_transform(train_x)

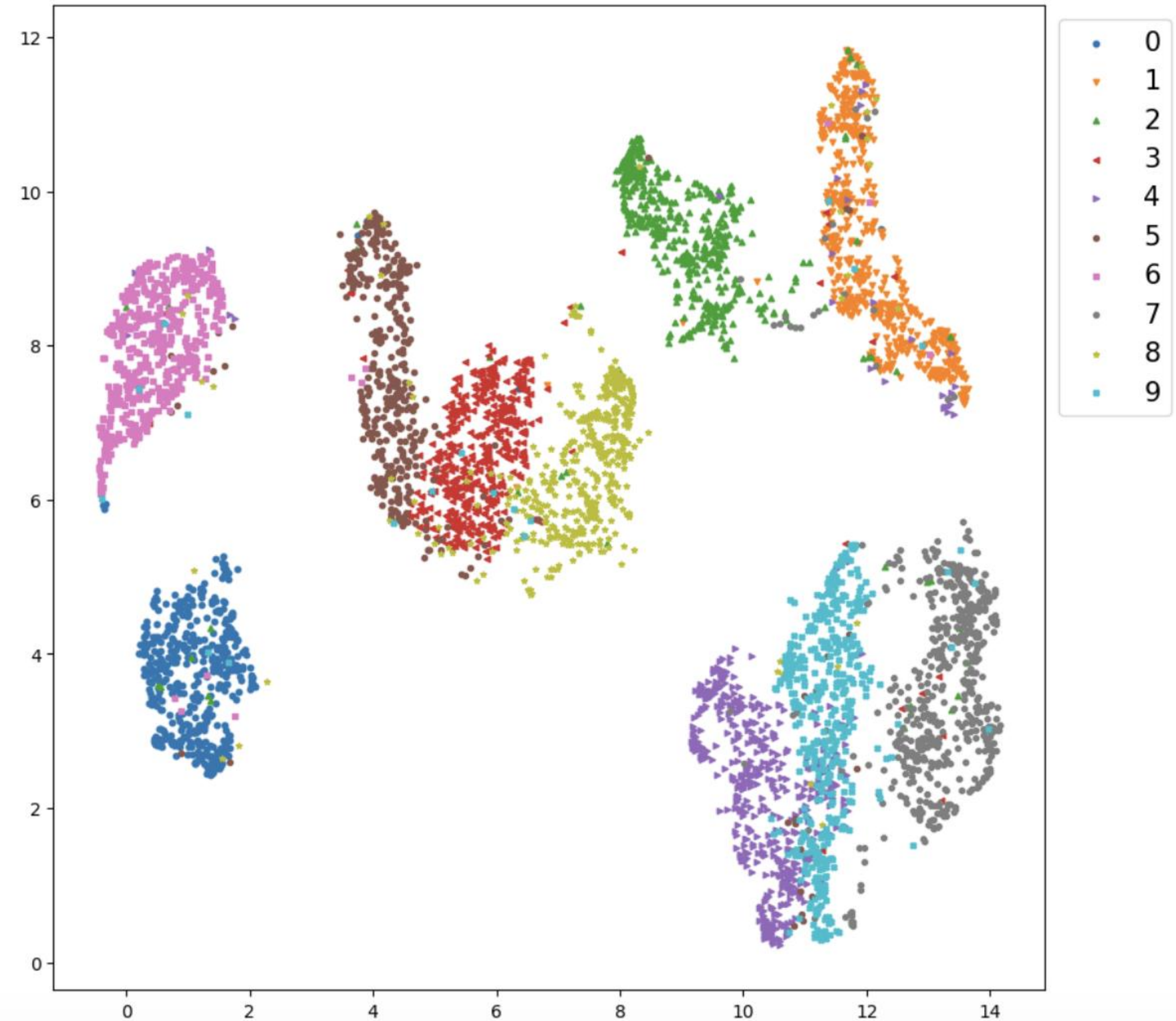
plt.figure(figsize=(10,10))
for i,marker in enumerate(markers):
    mask = train_y == i
    plt.scatter(x_tsne[mask, 0], x_tsne[mask, 1], label=i, s=10, alpha=1, marker=marker)
plt.legend(bbox_to_anchor=(1.00, 1), loc='upper left', fontsize=15)
```



Dimensionality Reduction for Beginners

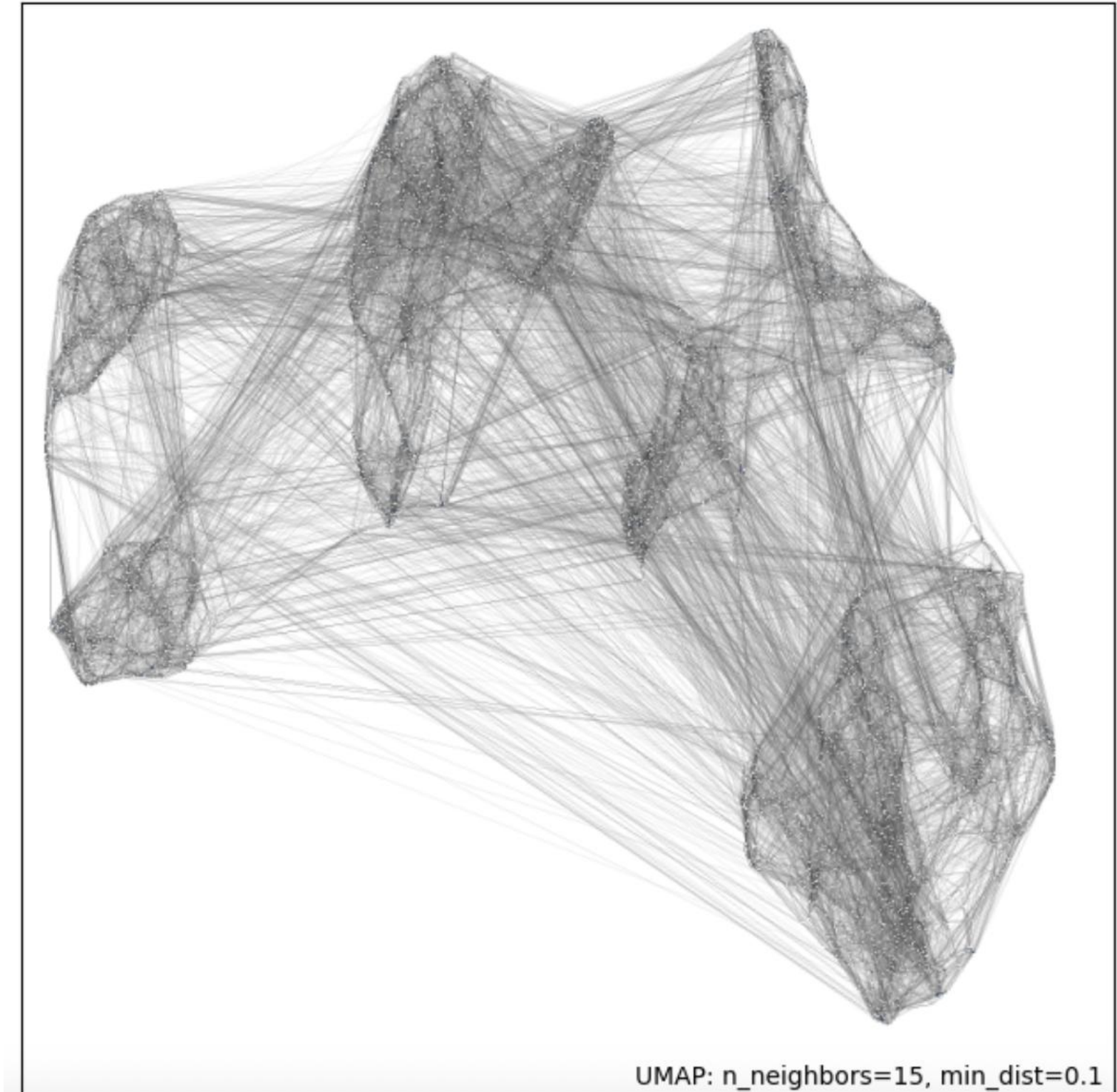
```
um = umap.UMAP()
x_umap = um.fit_transform(train_x)

plt.figure(figsize=(10,10))
for i, marker in enumerate(markers):
    plt.scatter(*x_umap[train_y == i].T, label=i, s=10, alpha=1, marker=marker)
plt.legend(bbox_to_anchor=(1.00, 1), loc='upper left', fontsize=15)
```



Dimensionality Reduction for Beginners

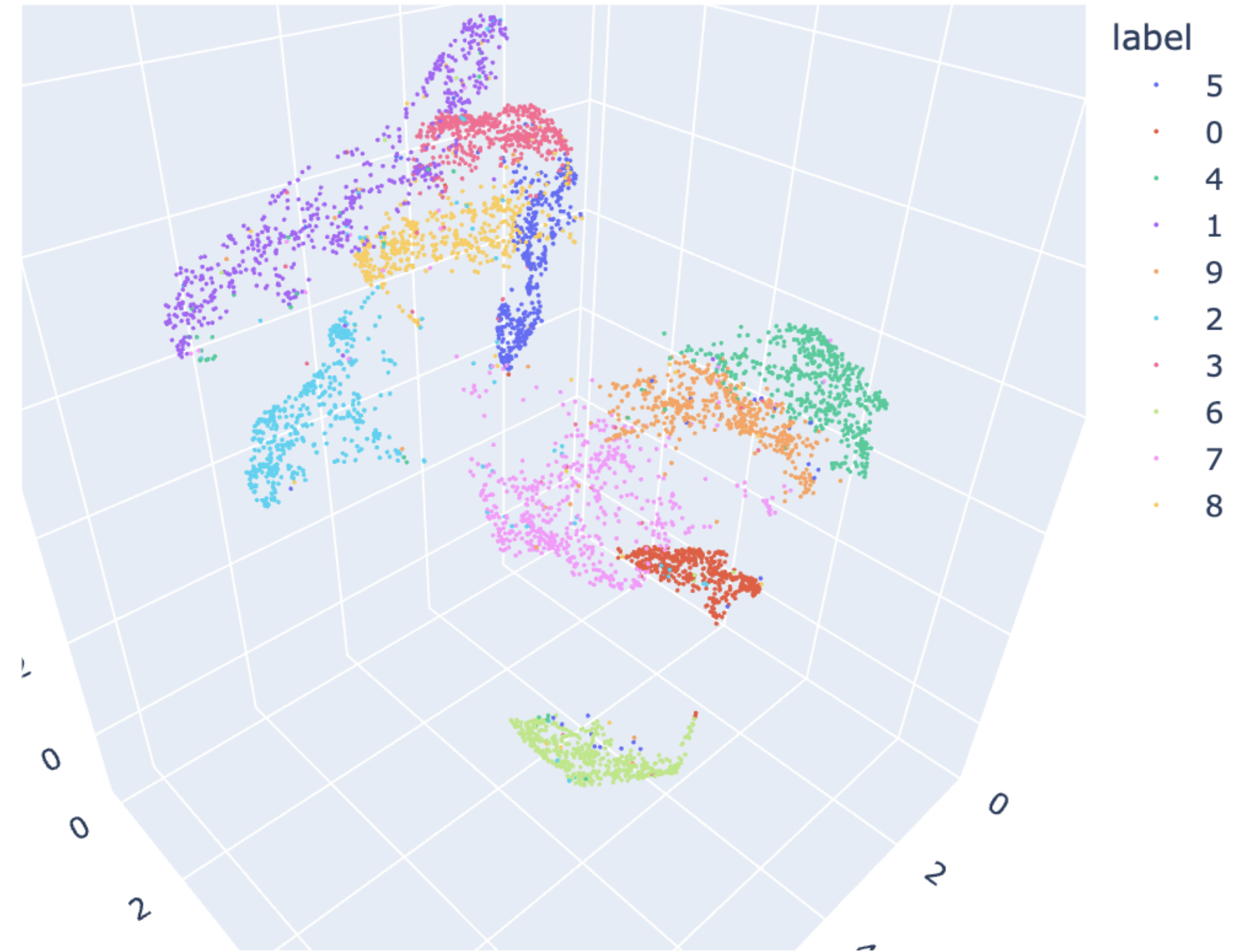
```
import umap.plot  
mapper = umap.UMAP().fit(train_x)  
umap.plot.connectivity(mapper, show_points=True)
```



Dimensionality Reduction for Beginners

```
umap_3d = UMAP(n_components=3, init='random', random_state=0)
x_umap = umap_3d.fit_transform(train_x)

new_df = pd.concat([pd.DataFrame(x_umap), pd.Series(train_y, name='label')], axis=1)
fig = px.scatter_3d(new_df, x=0, y=1, z=2, color='label', labels={'color': 'number'})
fig.update_traces(marker_size=1)
fig.show()
```



THANK YOU

